scan through ts and tell me whats the best possible way to make this.....as a website? as an app? if app then python? or c++? or java? or any other

Short answer: build it as a web application. Not a native app, and definitely not C++.

Why web

Look at who uses it: admin, internal staff, and customers (via a portal). That's three user groups on different devices, and customers can't be asked to install anything. A web app gives you one codebase, instant updates, role-based access in the browser, and nothing to install. The workload is also forms, tables, invoices, dashboards and background jobs, which is exactly what web stacks are built for.

A mobile app would only make sense later, as an optional customer-portal add-on. The responsive web portal already covers phones.

Why not C++ or Java (desktop/native)

- C++: built for performance-critical systems like games and engines. This system is CRUD, billing logic and reporting. C++ would multiply your development time for no benefit, and the "under 2 seconds with thousands of subscriptions" requirement is easily met by a normal database with proper indexes.
- Java: a solid enterprise choice (Spring Boot), and plenty of billing systems use it. But it's heavier and slower to build with. Worth it only if you're already a Java team.

Recommended stacks

Option 1: Python + Django (my top pick for this brief)

- Built-in auth, user roles/permissions, admin panel, password reset emails, and an ORM
- Django REST Framework for APIs, Celery + Redis for scheduled jobs (auto-generating invoices on billing day)
- PostgreSQL as the database
- Fastest path to a working, correct system

Option 2: JavaScript/TypeScript (Node.js + React)

- NestJS or Express backend, React frontend, PostgreSQL
- Best if you want a modern, highly interactive UI and one language across front and back end

Option 3: Java Spring Boot + React

- Best for strict enterprise environments, but more boilerplate

If this is a hackathon or time-limited project, go with Django + PostgreSQL + a simple React or even Django-template frontend.

Architecture that fits the requirements

Requirement in the brief	How to handle it
Unique email, strong password, reset link	Django auth + email backend with token links
Admin / Staff / Customer roles	Role-based access control (RBAC), with customers filtered to their own records
Products, variants (Odoo brand +560)	Product table + Variant table with price-extra field
Plans (daily/weekly/monthly/yearly, pause, renew, auto-close)	Plan model with flags; status field on subscriptions
Draft → Quotation → Confirmed → Active → Closed	State machine (e.g. django-fsm)
Quotation templates	Template table that copies into a new subscription
Auto invoicing, never twice, never skipped	Scheduled job (Celery beat/cron) + a unique constraint on subscription + billing period, wrapped in transactions
Auto taxes and discounts	Rules engine applied at invoice generation, with usage counters and date checks
Payments against invoices	Payment table linked to invoice; invoice flips to Paid when fully covered
Finance dashboard, filters, under 2s	Database indexes, pagination, aggregate queries, optional caching

The key technical decisions

1. **Idempotent billing:** this is the heart of "nothing billed twice or skipped." Use a unique key on (subscription, billing period) so a rerun can't duplicate.
2. **Use Decimal types for money**, never floats.
3. **Run billing as a background job**, not in a web request, since hundreds of subscriptions at once will time out otherwise.
4. **Indexes on status, due date, customer** to hit the two-second target.
5. **Audit trail:** log who changed what, since this is financial data.

Bottom line

Web app, Python (Django) + PostgreSQL + Celery, with a React or template-based frontend. It covers every scene in the brief with the least risk and effort.

If you tell me your team size, deadline, and what languages you already know, I can narrow this to one stack and sketch the database schema or a build plan.

its for a hackathon
team size 4 ppl
deadline is tdy at 5pm
what languages we already know doesnt rlly matter i think coz we can use ai i hope
also check ts out:-
Technology Purpose
React.js Frontend
Tailwind CSS Styling
JavaScript Programming logic
Supabase PostgreSQL Database
Supabase Auth Login and registration
Recharts Graphs and analytics
Lucide React Icons
React Router Page navigation

wud the layout suggested by you be better or this one?

Weighing Supabase's built-in auth, cron and access rules ▸

Use the stack they gave you. With a deadline of today at 5pm, it beats my Django suggestion, and I'd change my recommendation.

Why it wins for a one-day hackathon

- **Supabase Auth** gives you registration, login, unique emails and password reset emails out of the box. That's a whole scene from the brief done in about an hour.

- **Supabase Postgres + Row Level Security (RLS)** handles "customers see only their own subscriptions and invoices" and the admin/staff/customer roles at the database level, with no backend code.
- **No separate backend to build or deploy.** Django would cost you setup, hosting, Celery and Redis config, and CORS, which could eat half your day.
- **Recharts** covers the finance dashboard, **React Router** covers the portal vs admin pages, and Tailwind lets you build UI quickly.
- It's very AI-friendly. Claude or ChatGPT can generate React + Supabase code reliably.

Django is the better long-term architecture for billing, but you have hours, not weeks.

Where this stack is weak, and the workarounds

Problem	Hackathon fix
Auto-generating invoices on billing day	A Postgres function `generate_invoices()` plus a **"Run Billing" button** in the admin UI. If time allows, schedule it with `pg_cron`. A button demos well anyway.
Never billing twice	A **unique constraint on (subscription_id, billing_period)** in the invoices table
Taxes and discounts	Compute in the same Postgres function, or in a JS helper if you're short on time
Only admin creates staff and discounts	RLS policies keyed on a `role` column in a `profiles` table
Password rules (8+ chars, upper, lower, special)	Validate in the frontend form and set Supabase's minimum length in settings
Under 2 seconds	Add indexes on status, due_date and customer_id, and paginate tables

Suggested split for 4 people

1. **Person 1: Database and RLS.** Tables (profiles, products, variants, plans, subscriptions, subscription_lines, invoices, payments, discounts, taxes), constraints and roles. This is the critical path, so do it first.
2. **Person 2: Auth and portal.** Signup, login, reset password, role-based routing, and the customer portal (their subscriptions and invoices).
3. **Person 3: Admin and staff screens.** CRUD for products, plans, quotation templates, discounts, and the subscription status flow (Draft → Quotation → Confirmed → Active → Closed).
4. **Person 4: Billing and finance.** The invoice generation function, payments, and the Recharts dashboard (active subs, revenue, outstanding, overdue).

Timeline

- **First 1 to 2 hours:** agree on the schema, set up Supabase and the repo, and get everyone able to run the project.
- **Middle stretch:** build in parallel. Seed realistic fake data early so the dashboard has something to show.
- **Last 2 hours:** integrate, fix bugs, and rehearse the demo.

Tips

- **Cut scope aggressively.** Skip pause/renew edge cases, PDF printing and email sending unless you finish early.
- **Demo the story from the brief:** sign up → create product and plan → quote to subscription → click Run Billing → record payment → show the dashboard.
- **Seed data matters.** A dashboard with 200 fake subscriptions looks far better than an empty one.
- **Use one shared Supabase project** and keep the schema in a single SQL file in the repo.

I can write the full SQL schema (tables, constraints, RLS policies and the billing function) right now so Person 1 isn't starting from zero. Want that?

does this change ur suggestions? now tell me what the best web stack wud be

Weighing backend design against the hackathon's nice-to-have criteria ▸

Yes, it changes my recommendation. The "Nice to have" column rewards a real backend: your own APIs, data modelling, a local database, and not depending entirely on cloud tools. Supabase is cloud-hosted and does all of that for you, so it scores weakly on exactly those points. The "Must have" items (dynamic data, responsive UI, validation, navigation, Git) work with any stack.

Best stack for this

Layer	Choice
Frontend	React (Vite), Tailwind CSS, React Router, Recharts, Lucide React
Backend	**Node.js + Express** REST API
Database	**PostgreSQL running locally** (via Docker Compose, or installed directly)
ORM	**Prisma** (schema file, migrations, seed script)
Auth	JWT + bcrypt, with role middleware (admin / staff / customer)
Validation	**Zod** on the backend, the same rules mirrored in the forms
Billing job	**node-cron** plus a "Run Billing" button in the admin UI
Password reset	Nodemailer, with Ethereal or Mailtrap for testing, or print the reset link to the console as a local fallback

This keeps the stack you were given (React, Tailwind, Recharts, Lucide, React Router) and swaps Supabase for your own Express + Postgres backend. It stays JavaScript end to end, which is good if you're leaning on AI.

Why this fits the rules

- **Dynamic data:** everything comes from your API and database. The dashboard queries live data, and you can add polling every 30 seconds for a "real-time" feel.
- **Backend APIs, data modelling, local database:** you build all three, and Prisma's schema file shows off the data modelling.
- **Offline and local:** it runs fully on localhost with no internet needed, so the demo survives bad venue Wi-Fi.
- **Robust validation:** Zod schemas on the server (the real protection) plus matching client-side checks, covering the password rules, unique email, and discount and quantity limits.
- **Git properly:** the split below gives each person their own folders, so everyone commits meaningfully.

The tradeoff

Supabase would be faster, because auth, RLS and the database come ready-made. Your own backend costs roughly 1 to 2 extra hours of setup, and you have until 5pm. I think it's worth it for the judging criteria, but **don't spend more than the first hour on setup.** If you're stuck by then, fall back to Supabase.

Split for 4 people

1. **Auth + roles + portal:** register, login, reset password, role middleware, customer portal pages.
2. **Catalog:** products, variants, plans, quotation templates, and the admin and staff CRUD screens.
3. **Subscriptions + discounts + taxes:** the status flow (Draft → Quotation → Confirmed → Active → Closed) and the rules engine.
4. **Billing + payments + dashboard:** invoice generation, payments, and the Recharts finance view.

Each person works in their own `routes/`, `pages/` and `components/` folders on feature branches, and merges via pull requests. Everyone should make real commits, since the rules call it out.

Non-negotiables for the billing logic

- A **unique constraint on (subscriptionId, billingPeriod)** so billing can never run twice.
- Store **money as integers** (paise) or Prisma `Decimal`, never floats.
- **Indexes** on status, dueDate and customerId, with paginated tables, to hit the 2-second target.
- **Seed 100 to 200 fake subscriptions** so the dashboard looks real in the demo.

If you want, I can write the Prisma schema, the Express folder structure, and the billing function now, so everyone can start from the same base.

ok now give me the full descrpition of the project
its layout and web stack
task split into 4 ppl
and any other relevant info so that i can send it to my teamamtes

Make doc changes (batch) ▸
https://claude.ai/artifact/GrJNSkdW3qU2JJ2z2EdXV3

The team brief is ready. It covers the project overview, judging rules, tech stack, folder layout, UI layout, data model, business rules, API endpoints, the 4-person task split, timeline, Git workflow, demo script and risks. You can edit it inline, share the link with your teammates, or download it as PDF or Word.

Repo setup, Docker Compose Postgres, Prisma init, base Express server, shared Layout/Sidebar (do this first, others depend on it)
how do i do this

Weighing a Docker fallback for the database setup ▸
Here's the whole foundation in order. It should take 30 to 45 minutes. Do steps 1 to 6, push, and then tell the others to pull.

1. Repo and folders

Create an empty repo on GitHub, add all 3 teammates as collaborators, then:

```bash
git clone <repo-url> brightloop && cd brightloop
mkdir server client
```

Add a root `.gitignore`:

```
node_modules
.env
dist
```

2. Local Postgres with Docker

Create `docker-compose.yml` in the root:

```yaml
```

```
services:
  db:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_USER: brightloop
      POSTGRES_PASSWORD: brightloop
      POSTGRES_DB: brightloop
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

Run `docker compose up -d`. If Docker isn't installed or is a pain on someone's laptop, install Postgres directly, create a database called `brightloop`, and just change the connection string in `.env`.

3. Server and Prisma

bash

```
cd server
npm init -y
npm i express cors dotenv bcryptjs jsonwebtoken zod node-cron nodemailer @prisma/clien
npm i -D prisma@6 nodemon
npx prisma init
```

I pinned Prisma to version 6 on purpose. Version 7 changed its setup and will confuse everyone mid-hackathon.

In `server/package.json`, add `"type": "module"` and these scripts:

json

```
"scripts": {
  "dev": "nodemon src/index.js",
  "seed": "node prisma/seed.js"
},
"prisma": { "seed": "node prisma/seed.js" }
```

Create `server/.env` and also commit a copy as `.env.example`:

```
DATABASE_URL="postgresql://brightloop:brightloop@localhost:5432/brightloop"
JWT_SECRET="change-me"
PORT=5000
```

Start `prisma/schema.prisma` with just a User table. The team will agree on the rest together in the schema meeting (the data model is in the brief):

prisma

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum Role {
  ADMIN
  STAFF
  CUSTOMER
}

model User {
  id               String    @id @default(uuid())
  email            String    @unique
  passwordHash     String
  name             String
  role             Role      @default(CUSTOMER)
  resetToken       String?
  resetTokenExpiry DateTime?
  createdAt        DateTime  @default(now())
}
```

Then run `npx prisma migrate dev --name init`.

4. Base Express server

`server/src/index.js`:

```js
import 'dotenv/config';
import express from 'express';
import cors from 'cors';
import { PrismaClient } from '@prisma/client';

export const prisma = new PrismaClient();
const app = express();

app.use(cors({ origin: 'http://localhost:5173' }));
app.use(express.json());

app.get('/api/health', async (req, res) => {
  const users = await prisma.user.count();
  res.json({ status: 'ok', users });
});

// Each person mounts their routes here, e.g.:
// app.use('/api/auth', authRoutes);
// app.use('/api/products', productRoutes);

app.listen(process.env.PORT || 5000, () =>
  console.log(`API running on port ${process.env.PORT || 5000}`)
);
```

Create the empty folders `src/routes`, `src/middleware`, `src/services` and `src/schemas` (add a `.gitkeep` file in each so Git tracks them). Run `npm run dev` and open `http://localhost:5000/api/health`. If you see `{"status":"ok","users":0}`, the database connection works.

5. Client with Vite and Tailwind

```bash
```

```
cd ../client
npm create vite@latest . -- --template react
npm i
npm i react-router-dom recharts lucide-react tailwindcss @tailwindcss/vite
```

In `vite.config.js`:

```js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

Replace the contents of `src/index.css` with:

```css
@import "tailwindcss";
```

Create `src/api/client.js` so everyone calls the API the same way:

```js
const BASE = 'http://localhost:5000/api';

export async function api(path, options = {}) {
  const token = localStorage.getItem('token');
  const res = await fetch(BASE + path, {
    ...options,
    headers: {
      'Content-Type': 'application/json',
      ...(token && { Authorization: `Bearer ${token}` }),
      ...options.headers,
    },
    body: options.body ? JSON.stringify(options.body) : undefined,
  });
  const data = await res.json().catch(() => ({}));
  if (!res.ok) throw new Error(data.error || 'Request failed');
  return data;
}
```

6. Shared Layout and Sidebar

`src/components/Layout.jsx`. The menu depends on the role, and for now the role is read from localStorage until the auth flow is built:

```jsx
```

```
import { NavLink, Outlet } from 'react-router-dom';
import { LayoutDashboard, Users, Repeat, FileText, CreditCard,
  Package, ListChecks, Percent, Tag, UserCog, LogOut } from 'lucide-react';

const MENUS = {
  ADMIN: [
    ['/dashboard', 'Dashboard', LayoutDashboard],
    ['/customers', 'Customers', Users],
    ['/subscriptions', 'Subscriptions', Repeat],
    ['/invoices', 'Invoices', FileText],
    ['/payments', 'Payments', CreditCard],
    ['/products', 'Products', Package],
    ['/plans', 'Plans', ListChecks],
    ['/taxes', 'Taxes', Percent],
    ['/discounts', 'Discounts', Tag],
    ['/staff', 'Staff', UserCog],
  ],
  STAFF: [
    ['/dashboard', 'Dashboard', LayoutDashboard],
    ['/customers', 'Customers', Users],
    ['/subscriptions', 'Subscriptions', Repeat],
    ['/invoices', 'Invoices', FileText],
    ['/payments', 'Payments', CreditCard],
  ],
  CUSTOMER: [
    ['/portal/subscriptions', 'My Subscriptions', Repeat],
    ['/portal/invoices', 'My Invoices', FileText],
  ],
};

export default function Layout() {
  const user = JSON.parse(localStorage.getItem('user') || '{"name":"Demo","role":"ADMI
  const items = MENUS[user.role] || [];

  return (
    <div className="flex min-h-screen bg-slate-50">
      <aside className="hidden w-60 flex-col bg-slate-900 p-4 text-slate-200 md:flex">
        <h1 className="mb-6 px-2 text-xl font-bold text-white">Brightloop</h1>
        <nav className="flex flex-1 flex-col gap-1">
          {items.map(([to, label, Icon]) => (
            <NavLink key={to} to={to}
              className={({ isActive }) =>
                `flex items-center gap-3 rounded-lg px-3 py-2 text-sm ${
                  isActive ? 'bg-indigo-600 text-white' : 'hover:bg-slate-800'}`}>
              <Icon size={18} /> {label}
            </NavLink>
          ))}
        </nav>
      </aside>
      <div className="flex flex-1 flex-col">
        <header className="flex items-center justify-between border-b bg-white px-6 py
          <span className="text-sm text-slate-500">{user.name} · {user.role}</span>
          <button className="flex items-center gap-2 text-sm text-slate-600"
            onClick={() => { localStorage.clear(); window.location = '/login'; }}>
            <LogOut size={16} /> Logout
          </button>
        </header>
        <main className="flex-1 p-6"><Outlet /></main>
      </div>
    </div>
  );
}
```

Then wire up routing in `src/App.jsx`, with placeholder pages that each teammate replaces:

jsx

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import Layout from './components/Layout';

const Placeholder = ({ name }) => <h2 className="text-2xl font-semibold">{name}</h2>;

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/login" element={<Placeholder name="Login" />} />
        <Route element={<Layout />}>
          <Route path="/dashboard" element={<Placeholder name="Dashboard" />} />
          <Route path="/customers" element={<Placeholder name="Customers" />} />
          <Route path="/subscriptions" element={<Placeholder name="Subscriptions" />}
          <Route path="/invoices" element={<Placeholder name="Invoices" />} />
          <Route path="/payments" element={<Placeholder name="Payments" />} />
          <Route path="/products" element={<Placeholder name="Products" />} />
          <Route path="/plans" element={<Placeholder name="Plans" />} />
          <Route path="/taxes" element={<Placeholder name="Taxes" />} />
          <Route path="/discounts" element={<Placeholder name="Discounts" />} />
          <Route path="/staff" element={<Placeholder name="Staff" />} />
          <Route path="/portal/subscriptions" element={<Placeholder name="My Subscript
          <Route path="/portal/invoices" element={<Placeholder name="My Invoices" />}
        </Route>
        <Route path="*" element={<Navigate to="/dashboard" />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Run `npm run dev`, open `http://localhost:5173`, and you should see the sidebar with clickable placeholder pages.

7. Push and unblock the team

Add a short root `README.md` with the run steps:

```
1. docker compose up -d
2. cd server && cp .env.example .env && npm i && npx prisma migrate dev && npm run
dev
3. cd client && npm i && npm run dev
```

Then commit and push:

bash

```
git add .
git commit -m "chore: project foundation (express, prisma, vite, layout)"
git push origin main
```

Message the group: “Foundation is on main. Pull it, follow the README, and create your own feature branch.” After that, everyone replaces their placeholder pages and adds their own routes in `src/index.js`.

Common problems

- **Port 5432 already in use:** a local Postgres is running. Change the compose port to `"5433:5432"` and update `DATABASE_URL`.
- **CORS error in the browser:** the client must run on port 5173, or update the origin in `cors()`.
- **Prisma can’t connect:** run `docker ps` to check the container is up, and check the `.env` values.

While you do this, the other three should be agreeing on the full `schema.prisma` in a call, since that file is the one thing nobody can build without. Once you’ve done it, I can write the complete Prisma schema and the auth routes (register, login, password rules, reset) for you to paste in next.

lets go step by step
start with step 1 repo and folder explain what we are doing and why we're doing it coz i wanna learn too

Step 1: Repo and folders

What is a repo, and why do we need one?

A **repository** (repo) is a folder that Git tracks. Every change you make is saved as a snapshot (a *commit*), so you can see who changed what and go back if something breaks. **GitHub** hosts a copy online so all 4 of you can share the same code.

Without it, you'd be sending zip files on WhatsApp and overwriting each other's work. The judges also check that everyone used Git properly, so this is part of the score.

Part A: Create the repo on GitHub (one person does this)

1. Go to github.com, click **New repository**.
2. Name it `brightloop`.
3. Choose **Private** (or Public, either is fine for a hackathon).
4. Tick **Add a README file**. This makes the repo non-empty, which makes cloning easier.
5. Click **Create repository**.

Part B: Add your teammates

On the repo page: **Settings → Collaborators → Add people**, then enter each teammate's GitHub username. They'll get an invite email and must accept it.

Why: only collaborators can push code. If you skip this, your teammates will get a "permission denied" error later.

Part C: Clone it to your laptop

Open a terminal and run:

```bash
git clone <repo-url> brightloop
cd brightloop
```

(Copy the URL from the green **Code** button on GitHub.)

What this does: `clone` downloads the repo to your computer and links it to GitHub, so `git push` and `git pull` know where to send and fetch code. `cd brightloop` moves you into that folder.

Part D: Create the two main folders

```bash
mkdir server client
```

Why two folders? Our project has two separate programs:

- `server/` is the backend: the Express API and the database logic. It runs on port 5000.
- `client/` is the frontend: the React website users see. It runs on port 5173.

They talk to each other over HTTP (the client sends requests like "give me all invoices" and the server replies with data). Keeping them in separate folders means each has its own `package.json` and its own dependencies, so they don't get tangled. It also helps your task split: people on backend work mostly in `server/`, and people on screens work mostly in `client/`, so you'll get far fewer merge conflicts.

Part E: Add a `.gitignore`

Create a file called `.gitignore` in the root `brightloop/` folder with this content:

```
node_modules
.env
dist
```

Why each line matters:

- `node_modules`: the huge folder of downloaded libraries. It can be thousands of files. Nobody

should commit it, because anyone can recreate it by running `npm install`.

- `.env`: holds secrets like the database password and JWT secret. Never put secrets on GitHub. We'll share a safe `.env.example` instead.
- `dist`: the built output of the frontend. It's generated, so it doesn't belong in the repo.

Check that it worked

Run `ls` (or `dir` on Windows). You should see `README.md`, `server`, `client`, and (with `ls -a`) `.gitignore`.

One catch: Git doesn't track **empty folders**. So `server/` and `client/` won't show up on GitHub until they contain a file. That's fine, because they'll fill up in the next steps, so don't push yet.

Quick recap

Action	Why
Create repo on GitHub	One shared home for the code
Add collaborators	So all 4 of you can push
Clone	Get a linked copy on your laptop
`server/` and `client/` folders	Separate backend and frontend
`.gitignore`	Keep junk and secrets out of Git

When you've done this, tell me and we'll move to **Step 2: Postgres with Docker.** If anything errors on the way, paste the message here and I'll help you fix it.

do i not push this?

Deciding what to commit and push for .gitignore.

Good question. I was too vague earlier. **You can push now, and pushing `.gitignore` early is actually a good idea.** The only thing that won't go up is the empty `server/` and `client/` folders, because Git ignores empty folders. `.gitignore` is a real file, so it will be pushed.

Pushing it early is useful because your teammates will clone the repo with the ignore rules already in place. That stops anyone from accidentally committing `node_modules` or `.env` later.

How to push

bash

```
git add .gitignore
git commit -m "chore: add .gitignore"
git push origin main
```

What each command does:

- `git add .gitignore` puts the file in the *staging area*, which is basically a "these changes are going into the next snapshot" list.
- `git commit -m "..."` saves the snapshot locally with a message describing it.
- `git push origin main` uploads your commits to GitHub (`origin` is the GitHub copy, `main` is the branch).

If you want the folders to show up on GitHub too

Put an empty placeholder file in each one:

bash

```
touch server/.gitkeep client/.gitkeep
git add .
git commit -m "chore: add server and client folders"
git push origin main
```

On Windows PowerShell, use `New-Item server/.gitkeep` and `New-Item client/.gitkeep` instead of `touch`.

`.gitkeep` is just a convention. The file does nothing, it only gives Git something to track so the folder exists. We'll delete it naturally once real files go in.

Do you need to push at all right now?

Not strictly. If you'd rather wait and do one bigger commit after the foundation is built, that works too. But pushing small and often is a habit worth building for this hackathon, and it gives you a clean first commit in your history.

Teammates can run `git pull` any time to get what you pushed.

When you're ready, say "next" and we'll do **Step 2: Postgres with Docker**.



this ok right? next step

Checking the folder listing and its sync icons.
Yes, that's exactly right: `.git`, `client`, `server`, `.gitignore` and `README`.

One thing to check: those blue circular arrows on the right are **OneDrive sync icons**, which means this folder is probably inside your OneDrive (Desktop, Documents, etc.). That causes trouble later, because `node_modules` has tens of thousands of tiny files and OneDrive will try to sync all of them. It makes everything slow and can lock files mid-install.

The fix is simple: move the `brightloop` folder somewhere outside OneDrive, like `C:\dev\brightloop`. Git doesn't care where the folder lives, so just cut and paste it. Do this now, before we install anything.

Step 2: Postgres with Docker

What and why

Our app needs a **database** to store users, plans, subscriptions, invoices and payments. We chose **PostgreSQL**, a real relational database, and we want it running **on your own laptop** (the judges' "local database" point).

Docker is a tool that runs software in an isolated box called a *container*. Instead of installing and configuring Postgres by hand (different on Windows, Mac and Linux), we say "run Postgres 16" and Docker does it identically on all 4 laptops. If something breaks, you delete the container and start fresh in seconds.

Docker Compose lets us describe that setup in a file (`docker-compose.yml`) and start it with one command. The file lives in the repo, so teammates get the same database setup just by pulling.

Part A: Check Docker is installed

```bash
docker --version
```

If you get a version number, Docker is installed. Make sure **Docker Desktop is open and running** (check for the whale icon in your system tray). If the command isn't found, install Docker Desktop from docker.com. If it's a pain on any laptop, we have a fallback at the end.

Part B: Create `docker-compose.yml`

In the root `brightloop/` folder (next to `.gitignore`), create `docker-compose.yml`:

```yaml
```

```
services:
  db:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_USER: brightloop
      POSTGRES_PASSWORD: brightloop
      POSTGRES_DB: brightloop
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Line by line:

- `services: db:` defines one container, which we named `db`.
- `image: postgres:16` is which software to run. Docker downloads it the first time.
- `restart: unless-stopped` starts the database again automatically if your laptop reboots.
- `environment:` sets up the first database: a user called `brightloop`, with password `brightloop`, and a database called `brightloop`. (Weak password is fine here because it's only on your laptop.)
- `ports: "5432:5432"` connects port 5432 on your laptop to port 5432 inside the container. This is how our server will reach the database at `localhost:5432`.
- `volumes: pgdata:` stores the data in a Docker-managed disk area. Without this, **all your data disappears** whenever the container is removed. With it, data survives restarts.

Part C: Start it

bash

```
docker compose up -d
```

`up` starts everything in the file, and `-d` means "detached" (run in the background, so your terminal stays free). The first run downloads Postgres, which takes a minute.

Part D: Verify it's running

bash

```
docker ps
```

You should see a container with `postgres:16` and status `Up`. To peek inside the database:

bash

```
docker compose exec db psql -U brightloop -d brightloop -c "SELECT 1;"
```

If it prints a table with `1`, the database is alive and accepting connections.

Useful commands to remember

Command	What it does
`docker compose up -d`	Start the database
`docker compose stop`	Stop it (data kept)
`docker compose down`	Remove the container (data still kept in the volume)
`docker compose down -v`	Remove everything **including data**, a full reset
`docker compose logs db`	See error messages

Common problems

- **"port is already allocated" or "address already in use":** you already have Postgres installed and running locally. Change the line to `"5433:5432"` in the compose file, and remember to use port **5433** in the connection string later.
- **"cannot connect to the Docker daemon":** Docker Desktop isn't running. Open it and wait

until it says it's running.

- **Docker is too heavy or won't install:** install Postgres directly from postgresql.org, create a database named `brightloop` with user `brightloop`, and skip this file. Everything else in the project stays the same.

Commit it

```bash
git add docker-compose.yml
git commit -m "chore: add docker compose for local postgres"
git push origin main
```

Tell me when `docker ps` shows the container running (or paste any error), and we'll move on to **Step 3: Server and Prisma.**

docker install link

https://www.docker.com/products/docker-desktop/

Click **Download for Windows** and pick the version that matches your laptop. Most are **AMD64**; Snapdragon/ARM laptops are rare.

Install tips

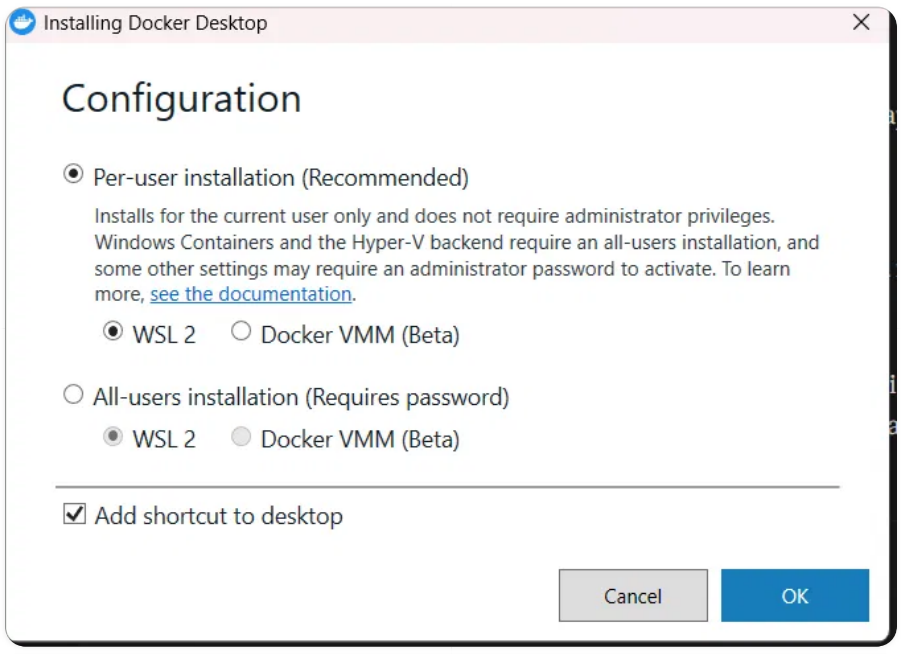
1. Run the installer and keep **"Use WSL 2 instead of Hyper-V"** ticked if it asks. WSL 2 is the Windows feature Docker runs on.
2. **Restart your laptop** when it finishes.
3. Open **Docker Desktop** and accept the terms. You can skip signing in.
4. Wait until the bottom-left says **Engine running** (green). This can take a minute.
5. Open a new terminal and check:

```bash
docker --version
```

If it gives you trouble

- **"WSL 2 installation is incomplete":** open PowerShell as admin, run `wsl --install`, then restart.
- **"Virtualization is disabled":** you need to turn on virtualization in your BIOS (look for "Intel VT-x" or "SVM Mode"). This takes a few minutes and varies by laptop, so search "enable virtualization [your laptop brand]".
- **Slow download or install:** don't burn more than 15 to 20 minutes on this. Install Postgres directly instead from https://www.postgresql.org/download/windows/. During setup, set the password to `brightloop`, keep port 5432, and afterwards create a database named `brightloop`. Everything else in the project stays the same.

Once `docker --version` works and Docker Desktop is running, go back to Step 2, Part B.



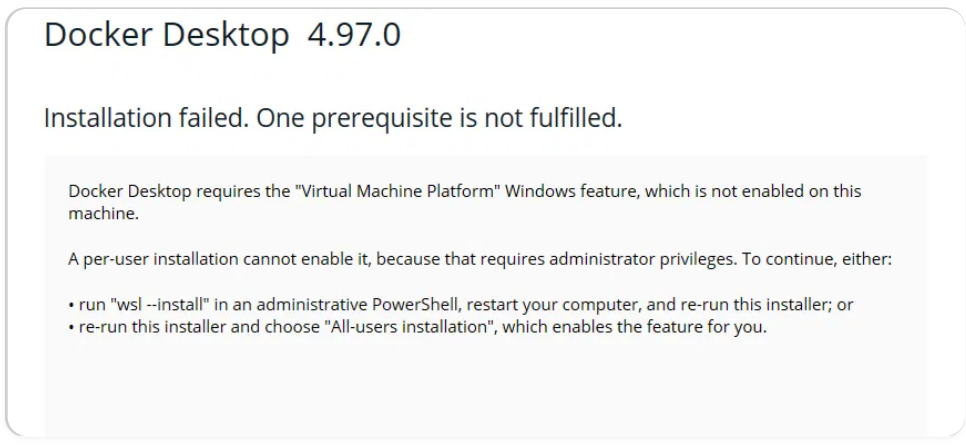
Leave it exactly as it is and click **OK**.

- **Per-user installation (Recommended)** is fine. It doesn't need an admin password, and it's all we need for this project.
- **WSL 2** is the right backend. Don't pick Docker VMM, which is still in beta.
- **Add shortcut to desktop** is optional, so keep it ticked if you like.

After you click OK, the installer will run for a few minutes. When it finishes:

1. It may ask you to **restart** the laptop, so do that.
2. Open **Docker Desktop** and accept the terms (you can skip signing in).
3. Wait for **Engine running** to show at the bottom left.
4. In a new terminal, run `docker --version`.

If it complains about WSL 2 or virtualization at any point, send me the message and I'll help you fix it.



This is a Windows permission issue, not a problem with your laptop. Docker needs the **Virtual Machine Platform** feature turned on, and a per-user install isn't allowed to enable it. You have two ways to fix it, and **Option A is the quickest**.

Option A: Re-run the installer as All-users

1. Close the error window.
2. Run the Docker installer again.
3. This time select **All-users installation (Requires password)** and keep **WSL 2** selected.
4. Click **OK**. Windows will ask for admin permission (click Yes, or enter the admin password).
5. When it finishes, **restart your laptop**.

The installer enables the missing Windows feature for you.

Option B: Enable it yourself with WSL

1. Click Start, type **PowerShell**, right-click it, and choose **Run as administrator**.
2. Run:

```powershell
wsl --install
```

3. Wait for it to finish, then **restart your laptop**.
4. Run the Docker installer again (the per-user option will work now).

After the restart

1. Open **Docker Desktop** and accept the terms. You can skip signing in.
2. Wait for **Engine running** at the bottom left.
3. In a new terminal, run `docker --version`.

If it still fails

If you see "virtualization is disabled" or something similar, virtualization is turned off in your BIOS. That takes a few extra minutes and depends on your laptop brand, so search "enable virtualization [your laptop brand]".

Time limit: if Docker isn't working within about 15 minutes, don't keep fighting it. Install Postgres directly from https://www.postgresql.org/download/windows/, set the password to `brightloop`, keep port 5432, and create a database named `brightloop`. The rest of the project works the same either way.

Tell me which option you go with, or paste any new error.

ukw give me a promp so that i can make all this in google ai studio typ shi Docker Compose Postgres, Prisma init, base Express server, shared Layout/Sidebar (do this first, others depend on it).

Drafting a prompt for AI Studio's app foundation.

Quick heads-up first: **AI Studio can generate the code, but it can't run Docker or Postgres for you.** Treat it as a file generator. Ask it for complete files with exact paths, then copy them into your local repo and run them on your laptop.

Paste this into AI Studio:

```
Generate the complete foundation code for a hackathon project called "Brightloop", a subscription management system. I will copy every file into my local repo and run it myself, so output each file in full, with its exact path as a heading above a code block. Do not skip files, do not use placeholders like "...rest of code", and do not build a preview app.

TECH STACK (use exactly this):
- Frontend: React (Vite, JavaScript, NOT TypeScript), Tailwind CSS v4 (via @tailwindcss/vite plugin), React Router v6/v7, Recharts, Lucide React
- Backend: Node.js + Express (ES modules, "type": "module"), cors, dotenv, bcryptjs, jsonwebtoken, zod, node-cron, nodemailer
- Database: PostgreSQL 16 running locally through Docker Compose
- ORM: Prisma version 6 (pin @prisma/client@6 and prisma@6, NOT version 7)

PROJECT STRUCTURE:
brightloop/
  docker-compose.yml
  README.md
  .gitignore
  server/
    package.json, .env.example
    prisma/schema.prisma
    src/index.js
    src/middleware/ (empty folder, add .gitkeep)
    src/routes/ (empty folder, add .gitkeep)
    src/services/ (empty folder, add .gitkeep)
    src/schemas/ (empty folder, add .gitkeep)
  client/
    package.json, vite.config.js, index.html
    src/main.jsx, src/App.jsx, src/index.css
    src/api/client.js
    src/components/Layout.jsx

REQUIREMENTS:

1. docker-compose.yml: one service "db" using postgres:16, user/password/database all "brightloop", port 5432:5432, restart unless-stopped, with a named volume "pgdata" so data persists.

2. server/package.json: "type": "module", scripts "dev" (nodemon src/index.js) and "seed" (node prisma/seed.js), all dependencies listed above, nodemon as devDependency.

3. server/.env.example with DATABASE_URL="postgresql://brightloop:brightloop@localhost:5432/brightloop", JWT_SECRET="change-me", PORT=5000.

4. server/prisma/schema.prisma: postgresql datasource using env("DATABASE_URL"), prisma-client-js generator, a Role enum (ADMIN, STAFF, CUSTOMER), and a User model with: id (uuid), email (unique), passwordHash, name, role (default CUSTOMER), resetToken (optional), resetTokenExpiry (optional DateTime), createdAt. Only the User model for now.

5. server/src/index.js: Express app that loads dotenv, creates and exports a PrismaClient, enables CORS for http://localhost:5173, uses express.json(), has a GET /api/health route returning { status: "ok", users: <user count> }, includes a clear comment block showing where teammates mount their routes (e.g. app.use('/api/auth', authRoutes)), and listens on process.env.PORT or 5000.

6. client/vite.config.js: react plugin + @tailwindcss/vite plugin.
   client/src/index.css: just @import "tailwindcss";

7. client/src/api/client.js: an exported async function api(path, options) that calls http://localhost:5000/api + path, sends JSON, attaches "Authorization: Bearer <token>" from localStorage "token" when present, and throws an Error with the server's error message when the response is not ok.
```

```
8. client/src/components/Layout.jsx: the shared app shell.
   - Dark slate left sidebar (hidden on mobile, with a working mobile menu button), a top header showing "<name> · <ROLE>" and a Logout button (clears localStorage and redirects to /login), and a main content area rendering <Outlet />.
   - Read the user from localStorage key "user" (JSON with name and role), defaulting to { name: "Demo", role: "ADMIN" }.
   - The sidebar menu depends on role, using NavLink with an active highlight (indigo) and Lucide icons:
     ADMIN: Dashboard, Customers, Subscriptions, Invoices, Payments, Products, Plans, Taxes, Discounts, Staff
     STAFF: Dashboard, Customers, Subscriptions, Invoices, Payments
     CUSTOMER: My Subscriptions (/portal/subscriptions), My Invoices (/portal/invoices)
   - Clean, consistent design: slate/indigo colour scheme, rounded corners, good spacing, responsive.

9. client/src/App.jsx: BrowserRouter with a /login placeholder route, and all the pages above as placeholder pages (a simple heading with the page name) nested inside the Layout route, plus a catch-all redirect to /dashboard.

10. .gitignore: node_modules, .env, dist.

11. README.md: short run instructions:
    1) docker compose up -d
    2) cd server && cp .env.example .env && npm i && npx prisma migrate dev --name init && npm run dev
    3) cd client && npm i && npm run dev

At the end, give me the exact terminal commands to run in order, and tell me what I should see at http://localhost:5000/api/health and http://localhost:5173 when it works.
```

How to use what it gives you

1. Copy each file into the matching path in your local `brightloop/` folder.
2. Follow the commands it prints at the end.
3. **Read through the code before committing.** The brief says not to blindly copy-paste AI output, so at least check it matches our folder names and that Prisma is pinned to version 6.

If AI Studio gives you a different Tailwind setup (like a `tailwind.config.js` with `@tailwind base`), that's the older v3 style. Either works, but don't mix the two.

If any file errors when you run it, paste the error here and I'll fix it.

no this is the full code right
i just want it to create the foundation

Checking whether the prompt produces full code.

Yes, that prompt only creates the foundation. It builds the Docker file, the Express server with a health check, Prisma with just the User table, and the sidebar layout with empty placeholder pages. No billing, subscriptions, or auth logic is in it. But it is long, so here's a shorter version:

```
Create the project foundation for "Brightloop", a subscription management hackathon app. Output every file in full with its exact path as a heading above each code block. No placeholders, no "rest of code here". Foundation only: no features, no business logic.

Stack: React (Vite, plain JavaScript), Tailwind CSS v4 (@tailwindcss/vite), React Router, Recharts, Lucide React on the frontend. Node.js + Express (ES modules), Prisma 6 (pin prisma@6 and @prisma/client@6), PostgreSQL 16 via Docker Compose on the backend.

Files needed:
- docker-compose.yml: postgres:16, user/password/db all "brightloop", port 5432, named volume for data
- server/package.json, server/.env.example (DATABASE_URL, JWT_SECRET, PORT=5000)
- server/prisma/schema.prisma: only a Role enum (ADMIN, STAFF, CUSTOMER) and a User model (id uuid, email unique, passwordHash, name, role, resetToken?, resetTokenExpiry?, createdAt)
- server/src/index.js: Express + CORS for http://localhost:5173 + GET /api/health returning { status: "ok", users: <count> }, with a comment showing where teammates mount their routes
- Empty folders (with .gitkeep): server/src/routes, middleware, services, schemas
- client/vite.config.js, client/src/index.css, client/src/main.jsx
- client/src/api/client.js: fetch wrapper that adds the JWT from localStorage and throws on errors
- client/src/components/Layout.jsx: responsive sidebar + top bar + <Outlet />. Slate/indigo theme, Lucide icons, role-based menu read from localStorage "user" (default { name: "Demo", role: "ADMIN" }):
  ADMIN: Dashboard, Customers, Subscriptions, Invoices, Payments, Products, Plans, Taxes, Discounts, Staff
  STAFF: Dashboard, Customers, Subscriptions, Invoices, Payments
  CUSTOMER: My Subscriptions, My Invoices
- client/src/App.jsx: routes for all the pages above as simple placeholder headings inside the Layout, plus a /login placeholder
- .gitignore (node_modules, .env, dist) and a short README with run steps

End with the exact terminal commands to run in order.
```

Don't worry if it adds extra small files, like `index.html`. Those are normal parts of a Vite project.

You can skip the docker-compose part of the prompt, since we already did that in Step 2. Just keep the file we made.

once it creates all those files tell me what I have to do

Planning the steps after generating files.

Once AI Studio gives you the files, follow these steps in order.

1. Put the files in the right places

Create each file at the exact path AI Studio gave it, inside your `brightloop/` folder. Make sure the folder is **outside OneDrive** (e.g. `C:\dev\brightloop`) and open it in VS Code.

Check that you end up with this:

```
brightloop/
├── docker-compose.yml
├── README.md
├── .gitignore
├── server/   (package.json, .env.example, prisma/schema.prisma, src/index.js, ...)
└── client/   (package.json, vite.config.js, index.html, src/..., ...)
```

If a file already exists (like `docker-compose.yml` or `.gitignore`), keep yours or overwrite it, either is fine.

2. Start the database

Open Docker Desktop, wait for **Engine running**, then in the root folder:

```bash
docker compose up -d
docker ps
```

You should see the postgres container as `Up`.

3. Set up and run the server

```bash
cd server
copy .env.example .env
npm install
npx prisma migrate dev --name init
npm run dev
```

(On Mac/Linux use `cp` instead of `copy`.)

What each does:

- `.env` is your private config copy (it's gitignored).
- `npm install` downloads the server's libraries.
- `prisma migrate dev` creates the `User` table in Postgres from `schema.prisma`.
- `npm run dev` starts the API.

Check: open `http://localhost:5000/api/health` in your browser. You should see `{"status":"ok","users":0}`. If so, the server and database are connected.

4. Set up and run the client

Open a **second terminal** (leave the server running):

```bash
cd client
npm install
npm run dev
```

Check: open `http://localhost:5173`. You should see the sidebar and top bar, and clicking menu items should switch between placeholder pages.

5. Quick review before committing

You're expected to understand what you pushed, so skim these:

- `server/package.json` has `"type": "module"`, and Prisma is version 6.
- `server/.env` is **not** showing in Git (run `git status`; if `.env` is listed, your `.gitignore` is wrong).
- Layout menu items match the brief (Admin, Staff, Customer menus).

6. Commit and push

From the root folder:

```bash
git add .
git commit -m "chore: project foundation (express, prisma, vite, layout)"
git push origin main
```

7. Tell the team

Send them this:

> Foundation is on main. Pull it and follow the README to run it. Then create your own branch: `git checkout -b feature/<yourname>-<thing>`. Replace your placeholder pages and add your routes in `server/src/index.js`.

If something breaks

Problem	Fix
`prisma migrate` says it can't reach the database	Docker isn't running, or `DATABASE_URL` in `.env` is wrong. Check `docker ps`.
Port 5432 already in use	Change the compose port to `5433:5432` and update `DATABASE_URL` to `localhost:5433`.
Page is blank or Tailwind styling is missing	Check that `index.css` has `@import "tailwindcss";`, that the plugin is in `vite.config.js`, and that `main.jsx` imports `index.css`.
`Cannot use import statement outside a module`	`"type": "module"` is missing from `server/package.json`.
Prisma version errors	Run `npm i prisma@6 @prisma/client@6` in `server/`.

Once the health check and the sidebar both work, you're done with the foundation. Paste any error you hit and I'll help fix it. After that, the next move is the full Prisma schema, which the whole team needs before they can build their parts.



Checking whether the remote already has a main branch ▸

Your local branch is called `master`, but you tried to push `main`. Git says "src refspec main does not match any" because there's no local branch named `main`.

Fix: rename your branch to `main`

```bash
git branch -M main
git push -u origin main
```

- `git branch -M main` renames your current branch to `main`, which matches GitHub's default.
- `-u` links your local `main` to `origin/main`, so from now on a plain `git push` and `git pull` work.

Before that, check you have a commit

If you haven't committed anything yet, there's nothing to push, and you'd get the same error. Check with:

```bash
git status
git log --oneline
```

If `git log` says there are no commits, do this first:

```bash
git add .
git commit -m "chore: project foundation"
git branch -M main
git push -u origin main
```

If the push is rejected with "fetch first" or "unrelated histories"

That happens when the GitHub repo already has a commit (for example, a README you created there) and your local folder was made with `git init` instead of `git clone`. Fix it with:

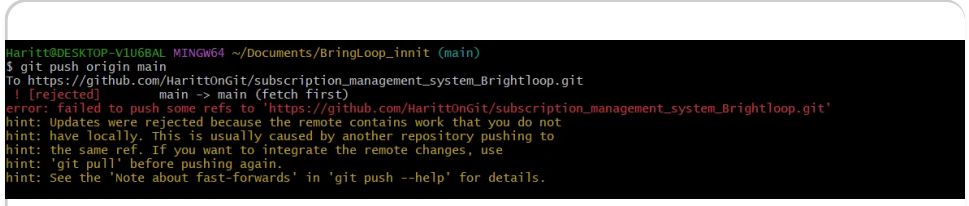
```bash
git pull origin main --allow-unrelated-histories
```

If it opens an editor for a merge message, just save and close it (in Vim: press `Esc`, type `:wq`, press Enter). If there's a conflict in `README.md`, keep either version. Then:

```bash
git push -u origin main
```

I noticed your folder is in `Documents`, which is often synced by OneDrive. Before you run `npm install`, move the project somewhere like `C:\dev` so OneDrive doesn't try to sync thousands of files.

Tell me what happens after the push, or paste the new error.



dude

This is the "fetch first" case I mentioned. The GitHub repo already has a commit (probably the README created with the repo), and your local folder doesn't have it, so Git won't let you overwrite it. Easy fix:

```bash
git pull origin main --allow-unrelated-histories
```

Then:

- **If an editor opens** asking for a merge message, just save and close it. In Vim, press `Esc`, type `:wq`, and press Enter.
- **If it says there's a conflict in** `README.md`, open the file and delete the lines containing `<<<<<<<`, `=======` and `>>>>>>>`. Keep whichever text you want, then run:

```bash
git add README.md
git commit -m "merge remote README"
```

Once the pull is done, push:

```bash
git push -u origin main
```

Why this happens: your local repo and the GitHub repo each started with their own first commit, so Git sees them as two unrelated histories. `--allow-unrelated-histories` tells Git to merge them anyway. You only need it this one time.

If you get a different error, paste it here.

so my github repo has some stufff alreadt but i want allat gone and just what i need in my local files in that repo how do i push it

Since the repo is brand new and nobody has pulled from it yet, you can safely **overwrite GitHub with your local files** using a force push:

```bash
git push -u origin main --force
```

That replaces whatever is on GitHub's `main` with your local `main`. The old stuff (probably just an auto-created README) is gone, and the repo will match your folder exactly.

Make sure you have a commit first. If you haven't committed yet, there's nothing to push:

```bash
git add .
git commit -m "chore: project foundation"
git push -u origin main --force
```

Why `--force` is okay now but not later

A normal push refuses to overwrite commits it doesn't recognise, to protect other people's work. `--force` skips that protection. It's fine today because you're the only one with anything in the repo. **Once teammates start pushing, never force-push to `main`**, because you could wipe out their commits. After this one push, use plain `git push`.

Check it worked

Refresh the GitHub page. You should see your `client/`, `server/`, `docker-compose.yml`, `.gitignore` and `README.md`. Also make sure `.env` and `node_modules` are **not** there. If they are, your `.gitignore` isn't working, so tell me and we'll fix it before your teammates clone.

If your teammates already accepted the invite and cloned the old version, tell them to run `git fetch origin && git reset --hard origin/main` (or just delete their folder and clone again).



my team meber already committed this tho

Weighing a force push against the teammate's commits ▸

Good thing you checked before force pushing. **Don't use `--force` now.** It would erase your teammate's commits, and you'd have an annoying argument and cleanup at the worst time.

Those files in the screenshot (`public`, `src`, `vite.config.js`, `eslint.config.js`, `package.json`) are a Vite React project sitting at the **root** of the repo. Ours should be inside `client/`. So the question is whether their work is worth keeping.

Step 1: Ask your teammate one thing

> "Is what you pushed just the default Vite template, or did you build real pages in it?"

Option A: It's just the template, or you don't need it

Both of you agree it can go. Tell them to stop pushing, then force push your version:

```bash
git push -u origin main --force
```

Afterwards, they run `git fetch origin && git reset --hard origin/main` (or delete their folder and re-clone). Only do this **with their agreement.**

Option B: Keep their work (safer)

Pull their commits into yours and move their files into `client/`:

```bash
git pull origin main --allow-unrelated-histories
```

Expect conflicts in `.gitignore` and `README.md`, since both histories have them. Open each file, delete the `<<<<<<<`, `=======` and `>>>>>>>` lines, and keep the combined content (for `.gitignore`, make sure it still has `node_modules`, `.env` and `dist`). Then:

```bash
git add .
git commit -m "merge remote work"
```

Now their root-level Vite files sit beside your `server/` and `client/` folders. If they contain real work, move it into the client:

```bash
git mv src client/src-theirs
```

Then copy over whatever you want and delete the rest. If it's only the template, just delete the duplicates:

```bash
git rm -r public src eslint.config.js index.html package.json package-lock.json vite.c
```

Careful: only run that `git rm` after you've confirmed your own `client/` folder has its own copies of those files. Otherwise you'll delete what you need.

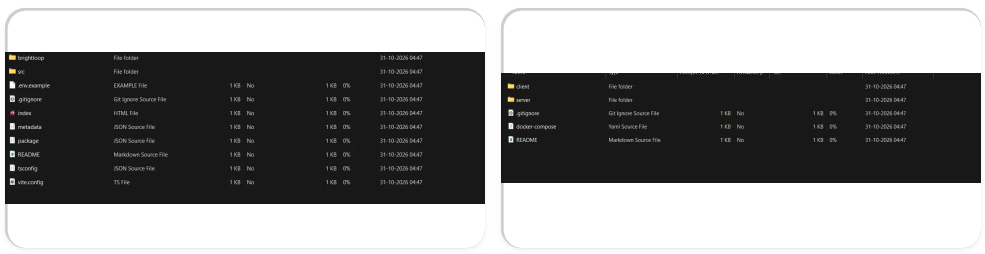
Then push normally:

```bash
git push -u origin main
```

Lesson for the rest of the day

Tell everyone: **nobody commits to `main` until the foundation is merged.** Once it's pushed, everyone pulls and works on their own `feature/...` branches. This kind of clash is exactly why we use branches.

Which option are you going with? If you tell me what's in their `src/`, I can say which one makes more sense.



is this crrct client and server inside brightloop folder

Checking the brightloop folder's contents against the default template ▸

Yes, image 2 is correct. Inside the `brightloop` folder you have `client/`, `server/`, `docker-compose.yml`, `.gitignore` and `README.md`, which is exactly our structure.

Ignore everything else in image 1

The files next to the `brightloop` folder (`src`, `index.html`, `package.json`, `tsconfig.json`, `vite.config.ts`, `metadata.json`, `.env.example`) are AI Studio's default starter app. They're TypeScript, and they're not part of our project. **Don't copy them into your repo.**

What to do

1. Copy **only the contents of the `brightloop` folder** (client, server, docker-compose.yml, .gitignore, README.md) into your Git repo folder (`BringLoop_innit`). Copy the contents, not the `brightloop` folder itself, or you'll end up with `repo/brightloop/client`.
2. Check the structure in your repo: `client/`, `server/` and `docker-compose.yml` should sit at the **top level** next to `.git`.
3. Quick sanity check: open `client/src` and make sure the files end in `.jsx` or `.js`, not `.tsx` or `.ts`. If you see TypeScript files, tell me, because our team plan is plain JavaScript.
4. Then run the foundation steps: Docker, `.env`, `npm install`, `prisma migrate`, and the two `npm run dev` commands.

The GitHub clash

Still unresolved from before: your teammate's template files are on GitHub's `main`. Ask them if it's just the default Vite template. If so, once you've confirmed your foundation runs locally,

commit it and do the **Option A force push** (with their OK). If they built real pages, use Option B.

Don't push yet. Get it running locally first, then we sort out the push.

why is it only showing readme bro

Narrowing down which cause fits the symptom ▸

Most likely cause: your files were never added to a commit, so GitHub only has the README. Git only uploads files that went through `git add` and `git commit`. Run these in your repo folder (the one with `.git`) and see what they say:

```bash
git status
git ls-files
git log --oneline
git branch
```

What the output tells you

- `git ls-files` shows only `README.md`: your other files aren't tracked. Fix:

```bash
git add .
git commit -m "chore: project foundation"
git push origin main
```

- `git status` says "Untracked files" or shows files in red: same fix as above, since the files were never added.
- `git status` says "nothing to commit" but GitHub still shows only the README: you haven't pushed, or you're looking at a different branch. Check that the branch dropdown on GitHub says `main`, and run `git push origin main`.
- `git ls-files` shows nothing for `client/` and `server/`: those folders may be empty. Git ignores empty folders, so make sure the files actually exist inside them (`ls client server`).
- The files are in a nested folder like `BringLoop_innit/brightloop/client`: you copied the `brightloop` folder itself, not its contents. Move `client`, `server` and `docker-compose.yml` up one level, next to `.git`.

Also check `.gitignore`

If `git add .` still skips everything, your `.gitignore` might have a stray line like `*` or `client` in it. It should only contain:

```
node_modules
.env
dist
```

If the push is rejected again, remember the remote already has your teammate's commits, so use the pull or force-push options from before (with their OK).

Paste the output of `git status` and `git ls-files` here and I can tell you exactly what's wrong.

for the 2nd team memeber tell which all files to change, in which folder and what to add basically just tell what all to do in detail, dont make a document bro

Choosing the money field and the schema layout for the catalog models ▸

Here's everything Person 2 (Catalog) needs: Products, Variants, Plans and Quotation Templates. Send it as is.

0. Before you start

```bash
```

```
git pull origin main
git checkout -b feature/p2-catalog
```

Only touch the files listed below. That keeps merge conflicts away from the other 3 people.

Files you own (create these):

```
server/src/routes/products.js
server/src/routes/plans.js
server/src/routes/templates.js
server/src/schemas/catalog.js
client/src/components/Table.jsx
client/src/components/Modal.jsx
client/src/components/FormField.jsx
client/src/pages/admin/Products.jsx
client/src/pages/admin/Plans.jsx
client/src/pages/admin/Templates.jsx
```

Shared files you only edit a little (tell the group when you do):

```
server/prisma/schema.prisma   (only ADD your models at the bottom)
server/src/index.js           (only ADD 3 import lines + 3 app.use lines)
client/src/App.jsx            (only swap your 3 placeholder routes)
client/src/components/Layout.jsx (add "Templates" to the ADMIN and STAFF menus if
missing)
```

Do the shared components (Table, Modal, FormField) first and push them within the first hour. Persons 3 and 4 will reuse them, so push them to a small PR early.

1. Database: add to the bottom of `schema.prisma`

Don't edit the `User` model or anyone else's. Only append:

prisma

```
enum BillingPeriod {
  DAILY
  WEEKLY
  MONTHLY
  YEARLY
}

model Product {
  id         String           @id @default(uuid())
  name       String
  type       String
  salesPrice Decimal          @db.Decimal(10, 2)
  costPrice  Decimal          @db.Decimal(10, 2)
  notes      String?
  isActive   Boolean          @default(true)
  createdAt  DateTime         @default(now())
  variants   ProductVariant[]
  templateLines TemplateLine[]

  @@index([name])
}

model ProductVariant {
  id         String  @id @default(uuid())
  productId  String
  product    Product @relation(fields: [productId], references: [id], onDelete: Cascad
  attribute  String  // e.g. "Brand"
  value      String  // e.g. "Odoo"
  extraPrice Decimal @db.Decimal(10, 2) @default(0) // e.g. 560

  templateLines TemplateLine[]

  @@index([productId])
}

model Plan {
  id            String        @id @default(uuid())
  name          String
  price         Decimal       @db.Decimal(10, 2)
  billingPeriod BillingPeriod
  minQuantity   Int           @default(1)
  availableFrom DateTime
  availableTo   DateTime
  autoClose     Boolean       @default(false) // close itself when it ends
  allowClose    Boolean       @default(false) // can be closed early
  allowPause    Boolean       @default(false)
  allowRenew    Boolean       @default(false)
  isActive      Boolean       @default(true)
  createdAt     DateTime      @default(now())
  templates     QuotationTemplate[]

  @@index([billingPeriod])
}

model QuotationTemplate {
  id           String         @id @default(uuid())
  name         String
  planId       String
  plan         Plan           @relation(fields: [planId], references: [id])
  validityDays Int            @default(30)
  createdAt    DateTime       @default(now())
  lines        TemplateLine[]
}

model TemplateLine {
  id         String            @id @default(uuid())
```

```
  templateId String
  template   QuotationTemplate @relation(fields: [templateId], references: [id], onDel
  productId  String
  product    Product           @relation(fields: [productId], references: [id])
  variantId  String?
  variant    ProductVariant?   @relation(fields: [variantId], references: [id])
  quantity   Int               @default(1)
}
```

Then run `npx prisma migrate dev --name catalog`. **Warn the group before you push this.** Everyone must pull and run `npx prisma migrate dev` afterwards. If Person 3 or 4 also changed the schema, resolve the conflict by keeping both sets of models.

Money uses `Decimal` (never Float). Prisma sends Decimals as strings in JSON, so wrap them in `Number(...)` when you display them on the frontend.

2. Backend

`server/src/schemas/catalog.js` (Zod validation)

```
js
```

```
import { z } from 'zod';

export const productSchema = z.object({
  name: z.string().trim().min(2, 'Name is required'),
  type: z.string().trim().min(2, 'Type is required'),
  salesPrice: z.coerce.number().min(0, 'Sales price cannot be negative'),
  costPrice: z.coerce.number().min(0, 'Cost price cannot be negative'),
  notes: z.string().optional().nullable(),
});

export const variantSchema = z.object({
  attribute: z.string().trim().min(1, 'Attribute is required'),
  value: z.string().trim().min(1, 'Value is required'),
  extraPrice: z.coerce.number().default(0),
});

export const planSchema = z
  .object({
    name: z.string().trim().min(2, 'Name is required'),
    price: z.coerce.number().min(0),
    billingPeriod: z.enum(['DAILY', 'WEEKLY', 'MONTHLY', 'YEARLY']),
    minQuantity: z.coerce.number().int().min(1, 'Minimum quantity is 1'),
    availableFrom: z.coerce.date(),
    availableTo: z.coerce.date(),
    autoClose: z.boolean().default(false),
    allowClose: z.boolean().default(false),
    allowPause: z.boolean().default(false),
    allowRenew: z.boolean().default(false),
  })
  .refine((d) => d.availableTo >= d.availableFrom, {
    message: 'End date must be after start date',
    path: ['availableTo'],
  });

export const templateSchema = z.object({
  name: z.string().trim().min(2, 'Name is required'),
  planId: z.string().uuid('Pick a plan'),
  validityDays: z.coerce.number().int().min(1, 'At least 1 day'),
  lines: z
    .array(
      z.object({
        productId: z.string().uuid(),
        variantId: z.string().uuid().optional().nullable(),
        quantity: z.coerce.number().int().min(1),
      })
    )
    .min(1, 'Add at least one product line'),
});
```

server/src/routes/products.js

```
js
```

```
import { Router } from 'express';
import { prisma } from '../index.js';
import { requireAuth, requireRole } from '../middleware/auth.js'; // Person 1 builds t
import { productSchema, variantSchema } from '../schemas/catalog.js';

const router = Router();
const staffOrAdmin = [requireAuth, requireRole('ADMIN', 'STAFF')];
const adminOnly = [requireAuth, requireRole('ADMIN')];

// list with search + pagination (keeps pages fast)
router.get('/', staffOrAdmin, async (req, res) => {
  const page = Number(req.query.page) || 1;
  const limit = Number(req.query.limit) || 10;
  const search = req.query.search?.toString() || '';
  const where = search ? { name: { contains: search, mode: 'insensitive' } } : {};
  const [items, total] = await Promise.all([
    prisma.product.findMany({
      where, include: { variants: true },
      orderBy: { createdAt: 'desc' },
      skip: (page - 1) * limit, take: limit,
    }),
    prisma.product.count({ where }),
  ]);
  res.json({ items, total, page, pages: Math.ceil(total / limit) });
});

router.get('/:id', staffOrAdmin, async (req, res) => {
  const product = await prisma.product.findUnique({
    where: { id: req.params.id }, include: { variants: true },
  });
  if (!product) return res.status(404).json({ error: 'Product not found' });
  res.json(product);
});

router.post('/', adminOnly, async (req, res) => {
  const parsed = productSchema.safeParse(req.body);
  if (!parsed.success) return res.status(400).json({ error: parsed.error.issues[0].mes
  res.status(201).json(await prisma.product.create({ data: parsed.data }));
});

router.put('/:id', adminOnly, async (req, res) => {
  const parsed = productSchema.safeParse(req.body);
  if (!parsed.success) return res.status(400).json({ error: parsed.error.issues[0].mes
  res.json(await prisma.product.update({ where: { id: req.params.id }, data: parsed.da
});

router.delete('/:id', adminOnly, async (req, res) => {
  try {
    await prisma.product.delete({ where: { id: req.params.id } });
    res.json({ ok: true });
  } catch {
    res.status(400).json({ error: 'Cannot delete: product is used in a template or sub
  }
});

// variants
router.post('/:id/variants', adminOnly, async (req, res) => {
  const parsed = variantSchema.safeParse(req.body);
  if (!parsed.success) return res.status(400).json({ error: parsed.error.issues[0].mes
  res.status(201).json(
    await prisma.productVariant.create({ data: { ...parsed.data, productId: req.params
  );
});

router.delete('/variants/:variantId', adminOnly, async (req, res) => {
  await prisma.productVariant.delete({ where: { id: req.params.variantId } });
```

```
  res.json({ ok: true });
});

export default router;
```

`plans.js` and `templates.js` (same pattern)

Copy `products.js` and change:

`plans.js`

- Use `planSchema` and `prisma.plan`.
- Same 5 routes: GET list, GET one, POST, PUT, DELETE. GET is allowed for staff and admin, writes are admin only.
- **Auto-close logic:** in the GET list route, before querying, add:

```js
  await prisma.plan.updateMany({
    where: { autoClose: true, isActive: true, availableTo: { lt: new Date() } },
    data: { isActive: false },
  });
```

This closes expired auto-close plans whenever the list is read. It's simple and enough for the hackathon.

- Add `PATCH /:id/close`: only works if `plan.allowClose` is true, otherwise return 400 "This plan cannot be closed early". Sets `isActive: false`.

`templates.js`

- Use `templateSchema` and `prisma.quotationTemplate`.
- `GET` includes `{ plan: true, lines: { include: { product: true, variant: true } } }`.
- `POST` creates the template and its lines in one call:

```js
const { lines, ...data } = parsed.data;
await prisma.quotationTemplate.create({
  data: { ...data, lines: { create: lines } },
  include: { lines: true },
});
```

- `PUT` replaces lines: `deleteMany` the old lines, then `create` the new ones, inside `prisma.$transaction`.
- Person 3 will use `GET /api/templates/:id` to build a subscription from a template, so keep that response shape stable.

`server/src/index.js`: add these lines only

```js
import productRoutes from './routes/products.js';
import planRoutes from './routes/plans.js';
import templateRoutes from './routes/templates.js';

app.use('/api/products', productRoutes);
app.use('/api/plans', planRoutes);
app.use('/api/templates', templateRoutes);
```

If Person 1's auth middleware isn't ready yet: create a temporary stub so you can keep working, and don't commit it.

```js
// server/src/middleware/auth.js (temporary)
export const requireAuth = (req, res, next) => next();
export const requireRole = () => (req, res, next) => next();
```

Agree with Person 1 now that the exports are named exactly `requireAuth` and

`requireRole('ADMIN', 'STAFF')`, so nothing breaks at merge time.

Test your API with Thunder Client or Postman before building any UI.

3. Frontend

Shared components (push these first)

`client/src/components/Modal.jsx`

jsx

```jsx
import { X } from 'lucide-react';

export default function Modal({ open, title, onClose, children }) {
  if (!open) return null;
  return (
    <div className="fixed inset-0 z-50 flex items-center justify-center bg-black/40 p-
      <div className="max-h-[90vh] w-full max-w-lg overflow-y-auto rounded-xl bg-white
        <div className="mb-4 flex items-center justify-between">
          <h3 className="text-lg font-semibold">{title}</h3>
          <button onClick={onClose}><X size={20} /></button>
        </div>
        {children}
      </div>
    </div>
  );
}
```

`client/src/components/FormField.jsx`

jsx

```jsx
export default function FormField({ label, error, children }) {
  return (
    <label className="mb-3 block text-sm">
      <span className="mb-1 block font-medium text-slate-700">{label}</span>
      {children}
      {error && <span className="mt-1 block text-xs text-red-600">{error}</span>}
    </label>
  );
}

export const inputClass =
  'w-full rounded-lg border border-slate-300 px-3 py-2 text-sm focus:border-indigo-500
```

`client/src/components/Table.jsx`

jsx

```
export default function Table({ columns, rows, page, pages, onPage, empty = 'No record
  return (
    <div className="overflow-x-auto rounded-xl border bg-white">
      <table className="w-full text-left text-sm">
        <thead className="bg-slate-100 text-slate-600">
          <tr>{columns.map((c) => <th key={c.label} className="px-4 py-3 font-medium">
        </thead>
        <tbody>
          {rows.length === 0 && (
            <tr><td colSpan={columns.length} className="px-4 py-8 text-center text-sla
          )}
          {rows.map((r) => (
            <tr key={r.id} className="border-t">
              {columns.map((c) => <td key={c.label} className="px-4 py-3">{c.render ?
            </tr>
          ))}
        </tbody>
      </table>
      {pages > 1 && (
        <div className="flex items-center justify-end gap-3 border-t p-3 text-sm">
          <button disabled={page <= 1} onClick={() => onPage(page - 1)} className="dis
          <span>{page} / {pages}</span>
          <button disabled={page >= pages} onClick={() => onPage(page + 1)} className=
        </div>
      )}
    </div>
  );
}
```

Add `Table`, `Modal` and `FormField` to a PR and tell the group on the chat as soon as it's merged.

`client/src/pages/admin/Products.jsx` (the pattern for all 3 pages)

jsx

```
import { useEffect, useState } from 'react';
import { Plus, Trash2 } from 'lucide-react';
import { api } from '../../api/client';
import Table from '../../components/Table';
import Modal from '../../components/Modal';
import FormField, { inputClass } from '../../components/FormField';

const empty = { name: '', type: '', salesPrice: '', costPrice: '', notes: '' };

export default function Products() {
  const [data, setData] = useState({ items: [], page: 1, pages: 1 });
  const [search, setSearch] = useState('');
  const [page, setPage] = useState(1);
  const [form, setForm] = useState(null); // null = modal closed
  const [error, setError] = useState('');
  const [variant, setVariant] = useState({ attribute: '', value: '', extraPrice: '' })

  const load = () =>
    api(`/products?page=${page}&search=${encodeURIComponent(search)}`).then(setData).c

  useEffect(() => { load(); }, [page, search]);

const save = async (e) => {
  e.preventDefault();
  setError('');
  try {
    if (form.id) await api(`/products/${form.id}`, { method: 'PUT', body: form });
    else await api('/products', { method: 'POST', body: form });
    setForm(null);
    load();
  } catch (err) { setError(err.message); }
};

const addVariant = async () => {
  try {
    await api(`/products/${form.id}/variants`, { method: 'POST', body: variant });
    setVariant({ attribute: '', value: '', extraPrice: '' });
    const fresh = await api(`/products/${form.id}`);
    setForm(fresh);
    load();
  } catch (err) { setError(err.message); }
};

const remove = async (id) => {
  if (!confirm('Delete this product?')) return;
  try { await api(`/products/${id}`, { method: 'DELETE' }); load(); }
  catch (err) { alert(err.message); }
};

return (
  <div>
    <div className="mb-4 flex flex-wrap items-center justify-between gap-3">
      <h2 className="text-2xl font-semibold">Products</h2>
      <div className="flex gap-2">
        <input className={inputClass} placeholder="Search..." value={search}
          onChange={(e) => { setSearch(e.target.value); setPage(1); }} />
        <button onClick={() => setForm(empty)}
          className="flex items-center gap-1 rounded-lg bg-indigo-600 px-4 py-2 text
          <Plus size={16} /> New
        </button>
      </div>
    </div>

    <Table
      page={data.page} pages={data.pages} onPage={setPage}
      rows={data.items}
```

```
        columns={[
          { label: 'Name', key: 'name' },
          { label: 'Type', key: 'type' },
          { label: 'Sales price', render: (r) => `₹${Number(r.salesPrice).toFixed(2)}`
          { label: 'Cost price', render: (r) => `₹${Number(r.costPrice).toFixed(2)}` }
          { label: 'Variants', render: (r) => r.variants.length },
          { label: '', render: (r) => (
            <div className="flex gap-3">
              <button className="text-indigo-600" onClick={() => setForm(r)}>Edit</but
              <button className="text-red-600" onClick={() => remove(r.id)}><Trash2 si
            </div>
          ) },
        ]}
      />

      <Modal open={!!form} title={form?.id ? 'Edit product' : 'New product'} onClose={
        {form && (
          <form onSubmit={save}>
            <FormField label="Name"><input className={inputClass} required value={form
            <FormField label="Type"><input className={inputClass} required value={form
            <div className="grid grid-cols-2 gap-3">
              <FormField label="Sales price"><input type="number" min="0" step="0.01"
              <FormField label="Cost price"><input type="number" min="0" step="0.01" c
            </div>
            <FormField label="Notes"><textarea className={inputClass} value={form.note
            {error && <p className="mb-3 text-sm text-red-600">{error}</p>}
            <button className="w-full rounded-lg bg-indigo-600 py-2 text-white">Save</

            {form.id && (
              <div className="mt-6 border-t pt-4">
                <h4 className="mb-2 font-medium">Variants</h4>
                {form.variants?.map((v) => (
                  <div key={v.id} className="mb-1 flex justify-between text-sm">
                    <span>{v.attribute}: {v.value} (+₹{Number(v.extraPrice)})</span>
                  </div>
                ))}
                <div className="mt-2 grid grid-cols-3 gap-2">
                  <input className={inputClass} placeholder="Brand" value={variant.att
                  <input className={inputClass} placeholder="Odoo" value={variant.valu
                  <input type="number" className={inputClass} placeholder="560" value=
                </div>
                <button type="button" onClick={addVariant} className="mt-2 text-sm tex
              </div>
            )}
          </form>
        )}
      </Modal>
    </div>
  );
}
```

`Plans.jsx` and `Templates.jsx`: same page structure, with these differences

Plans.jsx

- Table columns: Name, Price, Billing (Daily/Weekly/Monthly/Yearly), Min qty, Available from to, Status (Active or Closed), actions.
- Form fields: name, price, a `<select>` for billing period, min quantity (number, min 1), two date inputs (`type="date"`), and four checkboxes: **Auto-close at end date, Allow early close, Allow pause, Allow renew.**
- Date check on the client: if `availableTo < availableFrom`, show "End date must be after start date" before submitting.
- When editing, convert dates with `.slice(0, 10)` so the date input shows them.
- Show a "Close now" button in the row only if `plan.allowClose && plan.isActive`, calling `PATCH /plans/:id/close`.

Templates.jsx

- Table columns: Name, Plan, Validity (days), No. of lines, actions.

- Form fields: name, plan `<select>` (load options from `GET /plans`), validity days, and a **dynamic lines list:**
  - Each row has a product `<select>` (from `GET /products?limit=100`), a variant `<select>` (options come from the chosen product's `variants`), and a quantity input.
  - "+ Add line" appends a row, and a trash icon removes one.
  - Build `lines: [{ productId, variantId, quantity }]` in the body. Send `variantId: null` when none is chosen.
- Block saving with no lines ("Add at least one product line").

`client/src/App.jsx`: replace only your placeholder routes

```jsx
import Products from './pages/admin/Products';
import Plans from './pages/admin/Plans';
import Templates from './pages/admin/Templates';

<Route path="/products" element={<Products />} />
<Route path="/plans" element={<Plans />} />
<Route path="/templates" element={<Templates />} />
```

And in `Layout.jsx`, add `['/templates', 'Templates', FileStack]` (import `FileStack` from `lucide-react`) to the ADMIN menu. For STAFF, only add it if Person 3 needs staff to pick templates.

4. Rules to follow

- **Never** use Float for money. Use `Decimal` in Prisma and `Number()` in the UI.
- Validate in **both** places: the HTML attributes and the Zod schema (that's what the judges look for).
- Everything that lists data needs pagination and search.
- Variant pricing: final price of a line = product `salesPrice` + variant `extraPrice`. Person 3 will use this, so tell them.
- All writes (create, update, delete) are **admin only**. Staff can only read.

5. Interfaces other people depend on (don't change without telling them)

- `GET /api/products`: `{ items, total, page, pages }`, each item with `variants`.
- `GET /api/plans/:id`: full plan including the four flags.
- `GET /api/templates/:id`: template with `plan` and `lines` (each with `product` and `variant`). Person 3 uses this for "create subscription from template".

6. Checklist and Git

- [ ] Table, Modal and FormField pushed and merged early
- [ ] Product CRUD works, including variant with +560
- [ ] Plan CRUD works with date check and the four flags
- [ ] Auto-close works for expired plans
- [ ] Template CRUD works with lines
- [ ] Non-admins get 403 on write routes
- [ ] Looks fine on a phone-width screen
- [ ] Seed a few products, plans and templates and give them to Person 4 for `seed.js`

```bash
git add .
git commit -m "feat: products, variants, plans, templates (api + ui)"
git push origin feature/p2-catalog
```

Make small commits as you go (one per feature), then open a Pull Request on GitHub and ask a teammate to review and merge. If you get stuck on any piece, paste the error to me or to your AI tool along with the file.

for person 3 next

Covering the subscription status flow and filtered list endpoints ▸

Here's everything Person 3 needs: **Subscriptions, Taxes and Discounts**. Send it as is.

0. Before you start

```bash
git pull origin main
git checkout -b feature/p3-subscriptions
```

Order of work matters. You depend on Person 2's products, plans and templates, and on Person 1's auth. So start with the parts that need nothing from them:

1. Schema + migration
2. Taxes and Discounts (backend + UI)
3. `pricing.js` (Person 4 needs this the most)
4. Subscriptions (backend, then UI)

Files you own (create these):

```
server/src/routes/taxes.js
server/src/routes/discounts.js
server/src/routes/subscriptions.js
server/src/services/pricing.js
server/src/schemas/subscriptions.js
client/src/components/StatusBadge.jsx
client/src/pages/admin/Taxes.jsx
client/src/pages/admin/Discounts.jsx
client/src/pages/staff/Subscriptions.jsx
client/src/pages/staff/SubscriptionDetail.jsx
```

Shared files you edit a little (tell the group):

```
server/prisma/schema.prisma   (add your models + 4 one-line back-relations)
server/src/index.js           (add 3 imports + 3 app.use lines)
client/src/App.jsx            (swap your placeholder routes, add 2 new ones)
```

Reuse Person 2's `Table`, `Modal` and `FormField` components. Until they're merged, copy them from their branch or build a temporary version, and don't commit a duplicate.

1. Database

Append to `schema.prisma`:

```prisma
```

```
enum SubscriptionStatus {
  DRAFT
  QUOTATION
  CONFIRMED
  ACTIVE
  CLOSED
}

enum PaymentTerms {
  IMMEDIATE
  NET_15
  NET_30
  NET_60
}

enum DiscountType {
  FIXED
  PERCENT
}

enum DiscountAppliesTo {
  PRODUCT
  SUBSCRIPTION
}

model Tax {
  id         String   @id @default(uuid())
  name       String
  percentage Decimal  @db.Decimal(5, 2)
  isActive   Boolean  @default(true)
  createdAt  DateTime @default(now())
  lines      SubscriptionLine[]
}

model Discount {
  id          String            @id @default(uuid())
  name        String
  type        DiscountType
  value       Decimal           @db.Decimal(10, 2)
  minPurchase Decimal?          @db.Decimal(10, 2)
  minQuantity Int?
  startDate   DateTime
  endDate     DateTime
  usageLimit  Int?
  timesUsed   Int               @default(0)
  appliesTo   DiscountAppliesTo
  productId   String?
  product     Product?          @relation(fields: [productId], references: [id])
  isActive    Boolean           @default(true)
  createdAt   DateTime          @default(now())

  @@index([startDate, endDate])
}

model Subscription {
  id                 String             @id @default(uuid())
  number             String             @unique
  customerId         String
  customer           User               @relation(fields: [customerId], references: [
  planId             String
  plan               Plan               @relation(fields: [planId], references: [id])
  templateId         String?
  startDate          DateTime
  expiryDate         DateTime
  paymentTerms       PaymentTerms       @default(NET_30)
  status             SubscriptionStatus @default(DRAFT)
```

```
  quotationValidUntil DateTime?
  nextBillingDate     DateTime?
  createdAt           DateTime           @default(now())
  updatedAt           DateTime           @updatedAt
  lines               SubscriptionLine[]

  @@index([status])
  @@index([customerId])
  @@index([nextBillingDate])
}

model SubscriptionLine {
  id             String         @id @default(uuid())
  subscriptionId String
  subscription   Subscription   @relation(fields: [subscriptionId], references: [id],
  productId      String
  product        Product        @relation(fields: [productId], references: [id])
  variantId      String?
  variant        ProductVariant? @relation(fields: [variantId], references: [id])
  quantity       Int
  unitPrice      Decimal        @db.Decimal(10, 2)
  taxId          String?
  tax            Tax?           @relation(fields: [taxId], references: [id])
  amount         Decimal        @db.Decimal(10, 2) // quantity x unitPrice, before tax

  @@index([subscriptionId])
}
```

Prisma needs the other side of every relation. Add one line to each of these existing models:

- `User`: `subscriptions Subscription[]`
- `Plan`: `subscriptions Subscription[]`
- `Product`: `subscriptionLines SubscriptionLine[]` and `discounts Discount[]`
- `ProductVariant`: `subscriptionLines SubscriptionLine[]`

Those are Person 1's and Person 2's models, so tell them you're adding a line. **Person 4 adds `invoices Invoice[]` to Subscription themselves.** Don't add it, because Invoice doesn't exist yet and it would break validation.

Then run `npx prisma migrate dev --name subscriptions`. Tell the group to pull and migrate.

Decision to agree on with the team now: a subscription line's `unitPrice` = product `salesPrice` + variant `extraPrice` (editable by staff). The invoice is built from the lines. The plan decides **how often** to bill and the min quantity.

2. Validation: `server/src/schemas/subscriptions.js`

```
js
```

```
import { z } from 'zod';

// empty string / null -> null, otherwise a number (z.coerce would turn null into 0)
const optNum = (min = 0) =>
  z.preprocess((v) => (v === '' || v == null ? null : Number(v)), z.number().min(min).

export const taxSchema = z.object({
  name: z.string().trim().min(2, 'Name is required'),
  percentage: z.coerce.number().min(0, 'Min 0').max(100, 'Max 100'),
});

export const discountSchema = z
  .object({
    name: z.string().trim().min(2, 'Name is required'),
    type: z.enum(['FIXED', 'PERCENT']),
    value: z.coerce.number().positive('Value must be above 0'),
    minPurchase: optNum(0),
    minQuantity: optNum(1),
    startDate: z.coerce.date(),
    endDate: z.coerce.date(),
    usageLimit: optNum(1),
    appliesTo: z.enum(['PRODUCT', 'SUBSCRIPTION']),
    productId: z.string().uuid().optional().nullable(),
  })
  .refine((d) => d.endDate >= d.startDate, { message: 'End date must be after start da
  .refine((d) => d.type !== 'PERCENT' || d.value <= 100, { message: 'Percentage cannot
  .refine((d) => d.appliesTo !== 'PRODUCT' || d.productId, { message: 'Pick a product'

export const subscriptionSchema = z
  .object({
    customerId: z.string().uuid('Pick a customer'),
    planId: z.string().uuid('Pick a plan'),
    startDate: z.coerce.date(),
    expiryDate: z.coerce.date(),
    paymentTerms: z.enum(['IMMEDIATE', 'NET_15', 'NET_30', 'NET_60']),
    lines: z
      .array(
        z.object({
          productId: z.string().uuid(),
          variantId: z.string().uuid().optional().nullable(),
          quantity: z.coerce.number().int().min(1, 'Quantity must be at least 1'),
          taxId: z.string().uuid().optional().nullable(),
          unitPrice: optNum(0).optional(),
        })
      )
      .min(1, 'Add at least one line'),
  })
  .refine((d) => d.expiryDate > d.startDate, { message: 'Expiry must be after start da
```

3. Taxes and Discounts routes

Copy Person 2's `products.js` pattern:

- `taxes.js`: GET list and GET one (staff and admin), POST / PUT / DELETE (**admin only**), using `taxSchema` and `prisma.tax`. DELETE should catch errors and return "Tax is used in subscriptions".
- `discounts.js`: same, using `discountSchema` and `prisma.discount`. GET list includes `{ product: true }`. Admin only for writes, as the brief says only the admin creates discount rules.

Mount in `index.js`:

```
js
```

```
import taxRoutes from './routes/taxes.js';
import discountRoutes from './routes/discounts.js';
import subscriptionRoutes from './routes/subscriptions.js';

app.use('/api/taxes', taxRoutes);
app.use('/api/discounts', discountRoutes);
app.use('/api/subscriptions', subscriptionRoutes);
```

If Person 1's auth middleware isn't ready, use the temporary stub (`requireAuth` and `requireRole` that just call `next()`) and don't commit it. The export names must be exactly `requireAuth` and `requireRole('ADMIN', 'STAFF')`.

4. The pricing engine: `server/src/services/pricing.js`

This is the file Person 4 calls to build every invoice. Do it right and do it early. All maths is done in paise (integers) so there are no rounding bugs.

```
js
```

```
import { prisma } from '../index.js';

const toPaise = (v) => Math.round(Number(v) * 100);
const fromPaise = (p) => p / 100;

/**
 * Works out what an invoice for this subscription should contain.
 * Does NOT write anything. Person 4 calls this inside their billing transaction,
 * then calls markDiscountsUsed() with appliedDiscountIds.
 */
export async function computeInvoiceAmounts(subscription, asOf = new Date(), tx = pris
  const lines = await tx.subscriptionLine.findMany({
    where: { subscriptionId: subscription.id },
    include: { product: true, variant: true, tax: true },
  });

  const discounts = await tx.discount.findMany({
    where: { isActive: true, startDate: { lte: asOf }, endDate: { gte: asOf } },
  });
  const usable = discounts.filter((d) => d.usageLimit == null || d.timesUsed < d.usage

  const rows = lines.map((l) => ({ line: l, sub: toPaise(l.unitPrice) * l.quantity, di
  const applied = new Set();

  const amountOf = (d, base) =>
    d.type === 'PERCENT' ? Math.round((base * Number(d.value)) / 100) : toPaise(d.valu

  // 1) product-level discounts
  for (const d of usable.filter((x) => x.appliesTo === 'PRODUCT')) {
    for (const row of rows.filter((r) => r.line.productId === d.productId)) {
      if (d.minQuantity && row.line.quantity < d.minQuantity) continue;
      if (d.minPurchase && row.sub < toPaise(d.minPurchase)) continue;
      row.disc = Math.min(row.sub, row.disc + amountOf(d, row.sub));
      applied.add(d.id);
    }
  }

  // 2) whole-subscription discounts, spread across lines in proportion
  const totalQty = rows.reduce((s, r) => s + r.line.quantity, 0);
  const totalSub = rows.reduce((s, r) => s + r.sub, 0);
  for (const d of usable.filter((x) => x.appliesTo === 'SUBSCRIPTION')) {
    if (d.minQuantity && totalQty < d.minQuantity) continue;
    if (d.minPurchase && totalSub < toPaise(d.minPurchase)) continue;
    const remaining = rows.reduce((s, r) => s + (r.sub - r.disc), 0);
    if (remaining <= 0) continue;
    let amount = Math.min(amountOf(d, totalSub), remaining);
    let left = amount;
    rows.forEach((r, i) => {
      const room = r.sub - r.disc;
      const share = i === rows.length - 1 ? left : Math.min(room, Math.round((amount *
      r.disc += share;
      left -= share;
    });
    applied.add(d.id);
  }

  // 3) tax on (line amount - line discount)
  const out = rows.map((r) => {
    const taxable = r.sub - r.disc;
    const taxPercent = Number(r.line.tax?.percentage || 0);
    const taxAmt = Math.round((taxable * taxPercent) / 100);
    return {
      description: r.line.product.name + (r.line.variant ? ` (${r.line.variant.attribu
      productId: r.line.productId,
      quantity: r.line.quantity,
      unitPrice: Number(r.line.unitPrice),
```

```
      taxId: r.line.taxId,
      taxPercent,
      subtotal: fromPaise(r.sub),
      discountAmount: fromPaise(r.disc),
      taxAmount: fromPaise(taxAmt),
      lineTotal: fromPaise(taxable + taxAmt),
    };
  });

  const sum = (k) => out.reduce((s, l) => s + toPaise(l[k]), 0);
  return {
    lines: out,
    subtotal: fromPaise(sum('subtotal')),
    discountTotal: fromPaise(sum('discountAmount')),
    taxTotal: fromPaise(sum('taxAmount')),
    total: fromPaise(sum('lineTotal')),
    appliedDiscountIds: [...applied],
  };
}

/** Call once per invoice created, with computeInvoiceAmounts().appliedDiscountIds */
export async function markDiscountsUsed(ids, tx = prisma) {
  for (const id of ids) {
    await tx.discount.update({ where: { id }, data: { timesUsed: { increment: 1 } } })
  }
}
```

Self-check with this example: one line, qty 10, unit price ₹1000, 18% tax, and a 10% product discount. Expect subtotal 10000, discount 1000, tax 1620 (18% of 9000), total **10620**. If your output matches, the maths works.

5. Subscriptions routes: `server/src/routes/subscriptions.js`

```
js
```

```
import { Router } from 'express';
import { prisma } from '../index.js';
import { requireAuth, requireRole } from '../middleware/auth.js';
import { subscriptionSchema } from '../schemas/subscriptions.js';
import { computeInvoiceAmounts } from '../services/pricing.js';

const router = Router();
const staffOrAdmin = [requireAuth, requireRole('ADMIN', 'STAFF')];

const FLOW = {
  DRAFT: ['QUOTATION', 'CLOSED'],
  QUOTATION: ['CONFIRMED', 'DRAFT', 'CLOSED'],
  CONFIRMED: ['ACTIVE', 'CLOSED'],
  ACTIVE: ['CLOSED'],
  CLOSED: [],
};

const fullInclude = {
  customer: { select: { id: true, name: true, email: true } },
  plan: true,
  lines: { include: { product: true, variant: true, tax: true } },
};

async function nextNumber(tx) {
  const last = await tx.subscription.findFirst({ orderBy: { number: 'desc' }, select:
  const n = last ? parseInt(last.number.split('-')[1], 10) + 1 : 1;
  return `SUB-${String(n).padStart(5, '0')}`;
}

async function checkPlan(tx, planId, lines) {
  const plan = await tx.plan.findUnique({ where: { id: planId } });
  if (!plan || !plan.isActive) throw new Error('Plan is not available');
  const qty = lines.reduce((s, l) => s + l.quantity, 0);
  if (qty < plan.minQuantity) throw new Error(`This plan needs a minimum quantity of $
}

async function buildLines(tx, lines) {
  const products = await tx.product.findMany({ where: { id: { in: lines.map((l) => l.p
  const variants = await tx.productVariant.findMany({
    where: { id: { in: lines.map((l) => l.variantId).filter(Boolean) } },
  });
  return lines.map((l) => {
    const p = products.find((x) => x.id === l.productId);
    if (!p) throw new Error('Product not found');
    const v = l.variantId ? variants.find((x) => x.id === l.variantId) : null;
    if (l.variantId && (!v || v.productId !== p.id)) throw new Error('Variant does not
    const unitPrice = l.unitPrice ?? Number(p.salesPrice) + Number(v?.extraPrice || 0)
    return {
      productId: l.productId,
      variantId: l.variantId || null,
      quantity: l.quantity,
      unitPrice,
      taxId: l.taxId || null,
      amount: Math.round(unitPrice * l.quantity * 100) / 100,
    };
  });
}

// LIST (filters + pagination)
router.get('/', staffOrAdmin, async (req, res) => {
  // auto-close: active subscriptions past expiry whose plan has autoClose
  await prisma.subscription.updateMany({
    where: { status: 'ACTIVE', expiryDate: { lt: new Date() }, plan: { autoClose: true
    data: { status: 'CLOSED', nextBillingDate: null },
  });
```

```
const page = Number(req.query.page) || 1;
const limit = Number(req.query.limit) || 10;
const { status, customerId, search } = req.query;
const where = {
  ...(status && { status }),
  ...(customerId && { customerId }),
  ...(search && {
    OR: [
      { number: { contains: search, mode: 'insensitive' } },
      { customer: { name: { contains: search, mode: 'insensitive' } } },
    ],
  }),
};
const [items, total] = await Promise.all([
  prisma.subscription.findMany({
    where,
    include: { customer: { select: { id: true, name: true, email: true } }, plan: tr
    orderBy: { createdAt: 'desc' },
    skip: (page - 1) * limit,
    take: limit,
  }),
  prisma.subscription.count({ where }),
  ]);
  res.json({ items, total, page, pages: Math.ceil(total / limit) });
});

router.get('/:id', staffOrAdmin, async (req, res) => {
  const sub = await prisma.subscription.findUnique({ where: { id: req.params.id }, inc
  if (!sub) return res.status(404).json({ error: 'Subscription not found' });
  res.json(sub);
});

// Shows what the NEXT invoice would look like, with tax and discounts applied. Great
router.get('/:id/preview', staffOrAdmin, async (req, res) => {
  const sub = await prisma.subscription.findUnique({ where: { id: req.params.id } });
  if (!sub) return res.status(404).json({ error: 'Subscription not found' });
  res.json(await computeInvoiceAmounts(sub));
});

// CREATE (starts as DRAFT)
router.post('/', staffOrAdmin, async (req, res) => {
  const parsed = subscriptionSchema.safeParse(req.body);
  if (!parsed.success) return res.status(400).json({ error: parsed.error.issues[0].mes
  try {
    const sub = await prisma.$transaction(async (tx) => {
      const { lines, ...data } = parsed.data;
      await checkPlan(tx, data.planId, lines);
      return tx.subscription.create({
        data: { ...data, number: await nextNumber(tx), lines: { create: await buildLin
        include: fullInclude,
      });
    });
    res.status(201).json(sub);
  } catch (e) { res.status(400).json({ error: e.message }); }
});

// CREATE FROM QUOTATION TEMPLATE
router.post('/from-template/:templateId', staffOrAdmin, async (req, res) => {
  const { customerId, startDate, paymentTerms = 'NET_30' } = req.body;
  if (!customerId || !startDate) return res.status(400).json({ error: 'Customer and st
  const t = await prisma.quotationTemplate.findUnique({
    where: { id: req.params.templateId },
    include: { plan: true, lines: true },
  });
  if (!t) return res.status(404).json({ error: 'Template not found' });

  const start = new Date(startDate);
```

```
  const expiry = new Date(start); expiry.setFullYear(expiry.getFullYear() + 1);
  const validUntil = new Date(); validUntil.setDate(validUntil.getDate() + t.validityD
  const lines = t.lines.map((l) => ({ productId: l.productId, variantId: l.variantId,

  try {
    const sub = await prisma.$transaction(async (tx) => {
      await checkPlan(tx, t.planId, lines);
      return tx.subscription.create({
        data: {
          number: await nextNumber(tx), customerId, planId: t.planId, templateId: t.id
          startDate: start, expiryDate: expiry, paymentTerms, quotationValidUntil: val
          lines: { create: await buildLines(tx, lines) },
        },
        include: fullInclude,
      });
    });
    res.status(201).json(sub);
  } catch (e) { res.status(400).json({ error: e.message }); }
});

// EDIT (only while DRAFT)
router.put('/:id', staffOrAdmin, async (req, res) => {
  const parsed = subscriptionSchema.safeParse(req.body);
  if (!parsed.success) return res.status(400).json({ error: parsed.error.issues[0].mes
  const existing = await prisma.subscription.findUnique({ where: { id: req.params.id }
  if (!existing) return res.status(404).json({ error: 'Subscription not found' });
  if (existing.status !== 'DRAFT') return res.status(400).json({ error: 'Only draft su
  try {
    const sub = await prisma.$transaction(async (tx) => {
      const { lines, ...data } = parsed.data;
      await checkPlan(tx, data.planId, lines);
      await tx.subscriptionLine.deleteMany({ where: { subscriptionId: existing.id } })
      return tx.subscription.update({
        where: { id: existing.id },
        data: { ...data, lines: { create: await buildLines(tx, lines) } },
        include: fullInclude,
      });
    });
    res.json(sub);
  } catch (e) { res.status(400).json({ error: e.message }); }
});

// STATUS CHANGE (the state machine)
router.patch('/:id/status', staffOrAdmin, async (req, res) => {
  const { status } = req.body;
  const sub = await prisma.subscription.findUnique({
    where: { id: req.params.id },
    include: { plan: true, lines: true },
  });
  if (!sub) return res.status(404).json({ error: 'Subscription not found' });

  if (!FLOW[sub.status]?.includes(status))
    return res.status(400).json({ error: `Cannot move from ${sub.status} to ${status}`
  if (status === 'QUOTATION' && sub.lines.length === 0)
    return res.status(400).json({ error: 'Add at least one line first' });
  if (sub.status === 'QUOTATION' && status === 'CONFIRMED' && sub.quotationValidUntil
    return res.status(400).json({ error: 'This quotation has expired' });
  if (sub.status === 'ACTIVE' && status === 'CLOSED' && !sub.plan.allowClose)
    return res.status(400).json({ error: 'This plan cannot be closed early' });

  const data = { status };
  if (status === 'ACTIVE') data.nextBillingDate = sub.startDate; // Person 4's billing
  if (status === 'CLOSED') data.nextBillingDate = null;

  res.json(await prisma.subscription.update({ where: { id: sub.id }, data, include: fu
});
```

```
router.delete('/:id', staffOrAdmin, async (req, res) => {
  const sub = await prisma.subscription.findUnique({ where: { id: req.params.id } });
  if (!sub) return res.status(404).json({ error: 'Not found' });
  if (sub.status !== 'DRAFT') return res.status(400).json({ error: 'Only draft subscri
  await prisma.subscription.delete({ where: { id: sub.id } });
  res.json({ ok: true });
});

export default router;
```

6. Frontend

`client/src/components/StatusBadge.jsx` **(push early, Person 4 reuses it for invoices)**

jsx

```
const COLORS = {
  DRAFT: 'bg-slate-200 text-slate-700',
  QUOTATION: 'bg-blue-100 text-blue-700',
  CONFIRMED: 'bg-amber-100 text-amber-700',
  ACTIVE: 'bg-green-100 text-green-700',
  CLOSED: 'bg-red-100 text-red-700',
  PAID: 'bg-green-100 text-green-700',
  CANCELLED: 'bg-red-100 text-red-700',
  OVERDUE: 'bg-red-100 text-red-700',
};

export default function StatusBadge({ status }) {
  return (
    <span className={`rounded-full px-2.5 py-1 text-xs font-medium ${COLORS[status] ||
      {status}
    </span>
  );
}

const STEPS = ['DRAFT', 'QUOTATION', 'CONFIRMED', 'ACTIVE', 'CLOSED'];

export function Stepper({ status }) {
  const idx = STEPS.indexOf(status);
  return (
    <div className="flex flex-wrap items-center gap-2 text-xs">
      {STEPS.map((s, i) => (
        <div key={s} className="flex items-center gap-2">
          <span className={`rounded-full px-3 py-1 ${i <= idx ? 'bg-indigo-600 text-wh
          {i < STEPS.length - 1 && <span className="h-px w-6 bg-slate-300" />}
        </div>
      ))}
    </div>
  );
}
```

`Taxes.jsx` **and** `Discounts.jsx` **(admin pages)**

Same structure as Person 2's `Products.jsx` (table, search, New button, modal form, edit and delete).

- **Taxes:** columns are Name and Percentage. Form fields are name and a percentage input (`type="number" min="0" max="100" step="0.01"`).
- **Discounts:** columns are Name, Type, Value (show `10%` or `₹500`), Applies to, Valid from to, Used (`timesUsed / usageLimit` or `∞`), and a status that shows **Expired** or **Used up** when relevant. Form fields:
  - name, a type `<select>` (Fixed amount or Percentage), value
  - min purchase and min quantity (both optional)
  - start date and end date (`type="date"`)
  - usage limit (optional)
  - "Applies to" `<select>` (Specific product or Whole subscription). **Only show the product** `<select>` **when "Specific product" is chosen,** loading options from `GET /products?limit=100`.
  - Client checks before submit: end date is not before start, percentage is at most 100, and

a product is picked when required.
    - When editing, use `.slice(0, 10)` on dates for the date inputs.

`Subscriptions.jsx` (list, at `/subscriptions`)

- Top bar: search box (number or customer name), a status `<select>` filter (All + the 5 statuses), and two buttons: **New subscription** (goes to `/subscriptions/new`) and **From template** (opens a modal).
- The **From template** modal has a template `<select>` (`GET /templates`), a customer `<select>`, a start date, and payment terms. On submit it calls `POST /subscriptions/from-template/:id`, then navigates to `/subscriptions/${newId}`.
- Table columns: Number, Customer, Plan, Start, Expiry, Status (`<StatusBadge />`), and a View link. Use the shared `Table` with pagination (`page`, `pages`).
- Everyone on the team should be able to see where a subscription stands, and the badge column does that.

`SubscriptionDetail.jsx` (at `/subscriptions/:id` and `/subscriptions/new`)

Top section:

- Number (or "New subscription") with `<StatusBadge />`, then `<Stepper status={...} />` underneath.
- **Next action button**, based on this map:

```js
const NEXT = {
  DRAFT: ['QUOTATION', 'Send quotation'],
  QUOTATION: ['CONFIRMED', 'Confirm'],
  CONFIRMED: ['ACTIVE', 'Activate'],
  ACTIVE: ['CLOSED', 'Close subscription'],
};
```

It calls `PATCH /subscriptions/:id/status` with `{ status }`. Show the server's error message if it fails (like "This plan cannot be closed early").

- A small "Close / Cancel" button for DRAFT, QUOTATION and CONFIRMED that sends `{ status: 'CLOSED' }`.

Form (editable only when status is DRAFT or it's a new one; otherwise render everything read-only):

- Customer `<select>`, Plan `<select>` (`GET /plans`), start date, expiry date, and payment terms `<select>` with Immediate, Net 15, Net 30 and Net 60.
- Customers come from `GET /api/users?role=CUSTOMER&limit=100`. **Ask Person 1 to provide that route.** Until then, hardcode two fake customers.

Lines table (editable in DRAFT):

- Each row has a product `<select>`, a variant `<select>` (options come from the chosen product's `variants`), quantity, unit price (auto-filled as `salesPrice + extraPrice`, but editable), a tax `<select>` (`GET /taxes`), and the amount (`quantity x unitPrice`).
- "+ Add line" and a trash icon to remove a row.
- Below the table, a live subtotal.
- Validation on the client: at least one line, quantity at least 1, and the total quantity must meet `plan.minQuantity` (show the message "This plan needs a minimum quantity of N").

Bottom panel, **"Next invoice preview"**: when the subscription is saved, call `GET /subscriptions/:id/preview` and show lines, discount, tax and total. This is what shows the discount appearing by itself, and it's great for the demo.

`App.jsx`: replace only your routes

jsx

```
import Taxes from './pages/admin/Taxes';
import Discounts from './pages/admin/Discounts';
import Subscriptions from './pages/staff/Subscriptions';
import SubscriptionDetail from './pages/staff/SubscriptionDetail';

<Route path="/subscriptions" element={<Subscriptions />} />
<Route path="/subscriptions/new" element={<SubscriptionDetail />} />
<Route path="/subscriptions/:id" element={<SubscriptionDetail />} />
<Route path="/taxes" element={<Taxes />} />
<Route path="/discounts" element={<Discounts />} />
```

Use `useParams()` in the detail page. If there's no `id`, it's the "new" form. The `/subscriptions/new` route must sit above `/:id` so that "new" isn't treated as an id.

7. Contracts with the others (don't change without telling them)

Person 4 (billing) relies on:

- Billing picks subscriptions with `status = 'ACTIVE'` and `nextBillingDate <= today`.
- `nextBillingDate` is set to `startDate` when the subscription becomes ACTIVE. Person 4 moves it forward by the plan's period after each invoice.
- `computeInvoiceAmounts(subscription, asOf, tx)` returns `{ lines, subtotal, discountTotal, taxTotal, total, appliedDiscountIds }`. After saving the invoice, call `markDiscountsUsed(appliedDiscountIds, tx)`.
- `paymentTerms` is an enum. Mapping for due date: IMMEDIATE = 0 days, NET_15 = 15, NET_30 = 30, NET_60 = 60 (added to the invoice date).
- Auto-close: the subscription list auto-closes expired ACTIVE subscriptions on plans with `autoClose`. Person 4's billing should still bill any period that ended before the expiry date.

Person 1 (portal) relies on:

- The customer portal query is `prisma.subscription.findMany({ where: { customerId: req.user.id }, include: { plan: true, lines: { include: { product: true } } } })`. You don't build it, but tell them the shape.

Person 2 (catalog):

- You use `GET /api/templates/:id` and `GET /api/plans`. If they change those shapes, your template flow breaks.

8. Rules

- Money is **never Float**. Use `Decimal` in the database, paise inside `pricing.js`, and `Number()` on the frontend.
- Subscription status can only move along the allowed flow, and the server must enforce it (the UI buttons aren't enough).
- Writes to taxes and discounts are **admin only**. Staff can read them. Customers can't reach these routes at all.
- A discount must stop automatically when it's expired, inactive, or `timesUsed >= usageLimit`. That's handled in `pricing.js`, so don't add extra checks elsewhere.
- Every list needs pagination and search or filters.

9. Checklist

- [ ] Schema migrated and back-relations added
- [ ] Taxes CRUD works (admin only for writes)
- [ ] Discounts CRUD works with date, percentage and product validation
- [ ] `pricing.js` returns 10620 for the worked example
- [ ] Create subscription (manual) and from template
- [ ] Status flow works and blocks invalid jumps (try DRAFT to ACTIVE, it must fail)
- [ ] Expired quotation can't be confirmed
- [ ] Preview shows discount and tax automatically
- [ ] Auto-closed subscriptions show as CLOSED
- [ ] Looks fine on a phone-width screen
- [ ] Give Person 4 sample data for `seed.js`: taxes (GST 5%, 12%, 18%) and 2 discounts (10% on a product, ₹500 off a whole subscription)

10. Git

```
bash
```

```
git add .
git commit -m "feat: taxes, discounts, pricing engine"
git push origin feature/p3-subscriptions
```

Do small commits as each piece works (schema, taxes, discounts, pricing, subscriptions API, UI), and open a PR when `pricing.js` is done, so Person 4 can start using it while you finish the UI. Don't wait until the end for one giant commit. If anything breaks, paste the error and the file to me or your AI tool.

for person 4

Writing the git commit steps and print-layout notes ▸

Here's everything Person 4 needs: **Billing, Payments, Finance Dashboard and the seed data.** Send it as is.

0. Before you start

bash

```
git pull origin main
git checkout -b feature/p4-billing
```

You depend on Person 3's schema (Subscription, Tax, Discount) and on their `pricing.js`. Don't wait around for them:

1. Build the UI shells (Dashboard, Invoices, Payments) against fake data first.
2. As soon as Person 3's schema is merged, pull it and add your models.
3. Billing service, then invoice and payment routes, then wire up the UI.
4. The seed script has to be done by mid-afternoon, because the whole team needs real data for testing and the demo.

Files you own (create these):

```
server/src/services/billing.js
server/src/services/billingCron.js
server/src/routes/invoices.js
server/src/routes/payments.js
server/src/routes/billing.js
server/src/routes/dashboard.js
server/src/schemas/billing.js
server/prisma/seed.js
client/src/pages/staff/Invoices.jsx
client/src/pages/staff/InvoiceDetail.jsx
client/src/pages/staff/Payments.jsx
client/src/pages/dashboard/Dashboard.jsx
```

Shared files you edit a little (tell the group):

```
server/prisma/schema.prisma   (add your models + 2 one-line back-relations)
server/src/index.js           (add imports, 4 app.use lines, start the cron)
client/src/App.jsx            (swap your placeholder routes, add 1 route)
client/src/components/Layout.jsx (add "print:hidden" to the sidebar and header)
```

Reuse Person 2's `Table`, `Modal`, `FormField` and Person 3's `StatusBadge`. If they aren't merged yet, copy them locally and don't commit a duplicate.

1. Database

Append to `schema.prisma`:

prisma

```
enum InvoiceStatus {
  DRAFT
  CONFIRMED
  PAID
  CANCELLED
}

enum PaymentMethod {
  CASH
  CARD
  UPI
  BANK_TRANSFER
  CHEQUE
}

model Invoice {
  id             String        @id @default(uuid())
  number         String        @unique
  subscriptionId String
  subscription   Subscription  @relation(fields: [subscriptionId], references: [id])
  customerId     String
  customer       User          @relation(fields: [customerId], references: [id])
  periodStart    DateTime
  periodEnd      DateTime
  issueDate      DateTime
  dueDate        DateTime
  status         InvoiceStatus @default(DRAFT)
  subtotal       Decimal       @db.Decimal(12, 2)
  discountTotal  Decimal       @db.Decimal(12, 2) @default(0)
  taxTotal       Decimal       @db.Decimal(12, 2) @default(0)
  total          Decimal       @db.Decimal(12, 2)
  amountPaid     Decimal       @db.Decimal(12, 2) @default(0)
  sentAt         DateTime?
  createdAt      DateTime      @default(now())
  lines          InvoiceLine[]
  payments       Payment[]

  @@unique([subscriptionId, periodStart]) // the "never billed twice" guarantee
  @@index([status, dueDate])
  @@index([customerId])
  @@index([issueDate])
}

model InvoiceLine {
  id             String  @id @default(uuid())
  invoiceId      String
  invoice        Invoice @relation(fields: [invoiceId], references: [id], onDelete: Ca
  description    String
  productId      String?
  quantity       Int
  unitPrice      Decimal @db.Decimal(10, 2)
  subtotal       Decimal @db.Decimal(12, 2)
  discountAmount Decimal @db.Decimal(12, 2) @default(0)
  taxPercent     Decimal @db.Decimal(5, 2) @default(0)
  taxAmount      Decimal @db.Decimal(12, 2) @default(0)
  lineTotal      Decimal @db.Decimal(12, 2)

  @@index([invoiceId])
}

model Payment {
  id        String        @id @default(uuid())
  invoiceId String
  invoice   Invoice       @relation(fields: [invoiceId], references: [id])
  method    PaymentMethod
  amount    Decimal       @db.Decimal(12, 2)
```

```
  date      DateTime
  note      String?
  createdAt DateTime      @default(now())

  @@index([invoiceId])
  @@index([date])
}
```

Add one line to two existing models (tell the owners):

- `User`: `invoices Invoice[]`
- `Subscription`: `invoices Invoice[]`

Then run `npx prisma migrate dev --name billing` and tell the group to pull and migrate.

Why the design is like this:

- `@@unique([subscriptionId, periodStart])` means the database itself refuses a second invoice for the same subscription and period, even if the button is clicked twice.
- `InvoiceLine` stores a **snapshot** (description, price, tax %), so old invoices never change when a product or tax is edited later.
- `amountPaid` on the invoice is kept up to date on every payment, so the dashboard doesn't have to sum payments on every request. That's what keeps pages under 2 seconds.

2. Validation: `server/src/schemas/billing.js`

```js
import { z } from 'zod';

export const paymentSchema = z.object({
  invoiceId: z.string().uuid('Pick an invoice'),
  method: z.enum(['CASH', 'CARD', 'UPI', 'BANK_TRANSFER', 'CHEQUE']),
  amount: z.coerce.number().positive('Amount must be above 0'),
  date: z.coerce.date().refine((d) => d <= new Date(Date.now() + 86400000), 'Date cann
  note: z.string().trim().max(500, 'Note is too long').optional().nullable(),
});

export const invoiceStatusSchema = z.object({
  status: z.enum(['CONFIRMED', 'CANCELLED']),
});
```

3. The billing engine: `server/src/services/billing.js`

This is the heart of "nothing billed twice or skipped". Read it fully before you paste it.

```js
```

```
import { prisma } from '../index.js';
import { computeInvoiceAmounts, markDiscountsUsed } from './pricing.js'; // Person 3's

const DAY = 86400000;
const TERMS_DAYS = { IMMEDIATE: 0, NET_15: 15, NET_30: 30, NET_60: 60 };
const MAX_PERIODS_PER_RUN = 60; // safety cap so a bad date can't loop forever

const utcDay = (d) => new Date(Date.UTC(d.getUTCFullYear(), d.getUTCMonth(), d.getUTCD

export function addPeriod(date, period) {
  const y = date.getUTCFullYear(), m = date.getUTCMonth(), d = date.getUTCDate();
  switch (period) {
    case 'DAILY': return new Date(Date.UTC(y, m, d + 1));
    case 'WEEKLY': return new Date(Date.UTC(y, m, d + 7));
    case 'MONTHLY': {
      const last = new Date(Date.UTC(y, m + 2, 0)).getUTCDate(); // days in next month
      return new Date(Date.UTC(y, m + 1, Math.min(d, last)));
    }
    case 'YEARLY': {
      const last = new Date(Date.UTC(y + 1, m + 1, 0)).getUTCDate();
      return new Date(Date.UTC(y + 1, m, Math.min(d, last)));
    }
    default: throw new Error(`Unknown billing period ${period}`);
  }
}

async function nextInvoiceNumber(tx) {
  const last = await tx.invoice.findFirst({ orderBy: { number: 'desc' }, select: { num
  const n = last ? parseInt(last.number.split('-')[1], 10) + 1 : 1;
  return `INV-${String(n).padStart(5, '0')}`;
}

// Bills ONE period of ONE subscription, and moves nextBillingDate forward. All or not
async function billOnePeriod(sub, periodStart, autoConfirm) {
  const nextStart = addPeriod(periodStart, sub.plan.billingPeriod);

  return prisma.$transaction(async (tx) => {
    const existing = await tx.invoice.findUnique({
      where: { subscriptionId_periodStart: { subscriptionId: sub.id, periodStart } },
    });

    let created = false;
    if (!existing) {
      const amounts = await computeInvoiceAmounts(sub, new Date(), tx);
      if (amounts.lines.length === 0) throw new Error('Subscription has no lines');

      await tx.invoice.create({
        data: {
          number: await nextInvoiceNumber(tx),
          subscriptionId: sub.id,
          customerId: sub.customerId,
          periodStart,
          periodEnd: new Date(nextStart.getTime() - DAY),
          issueDate: periodStart,
          dueDate: new Date(periodStart.getTime() + TERMS_DAYS[sub.paymentTerms] * DAY
          status: autoConfirm ? 'CONFIRMED' : 'DRAFT',
          subtotal: amounts.subtotal,
          discountTotal: amounts.discountTotal,
          taxTotal: amounts.taxTotal,
          total: amounts.total,
          lines: {
            create: amounts.lines.map((l) => ({
              description: l.description, productId: l.productId, quantity: l.quantity
              unitPrice: l.unitPrice, subtotal: l.subtotal, discountAmount: l.discount
              taxPercent: l.taxPercent, taxAmount: l.taxAmount, lineTotal: l.lineTotal
            })),
```

```
          },
        },
      });
      await markDiscountsUsed(amounts.appliedDiscountIds, tx);
      created = true;
    }

    // move the clock forward (even if the invoice already existed, so the system heal
    const finished = nextStart > sub.expiryDate;
    await tx.subscription.update({
      where: { id: sub.id },
      data: {
        nextBillingDate: finished ? null : nextStart,
        ...(finished && sub.plan.autoClose ? { status: 'CLOSED' } : {}),
      },
    });
    return created;
  });
}

let running = false; // stops two runs at the same time inside this server

export async function runBilling({ asOf = new Date(), autoConfirm = false } = {}) {
  if (running) throw new Error('Billing is already running');
  running = true;
  const result = { subscriptionsChecked: 0, invoicesCreated: 0, skipped: 0, failed: []
  try {
    const today = utcDay(asOf);
    const due = await prisma.subscription.findMany({
      where: { status: 'ACTIVE', nextBillingDate: { lte: asOf } },
      include: { plan: true },
    });
    result.subscriptionsChecked = due.length;

    for (const sub of due) {
      let next = utcDay(sub.nextBillingDate);
      let guard = 0;
      try {
        // while loop = catch-up: if we missed 3 months, we bill all 3 (nothing skippe
        while (next <= today && next <= sub.expiryDate && guard++ < MAX_PERIODS_PER_RU
          try {
            const created = await billOnePeriod(sub, next, autoConfirm);
            created ? result.invoicesCreated++ : result.skipped++;
          } catch (e) {
            if (e.code === 'P2002') result.skipped++; // unique constraint: already bi
            else throw e;
          }
          next = addPeriod(next, sub.plan.billingPeriod);
        }
      } catch (e) {
        result.failed.push({ subscription: sub.number, error: e.message }); // one bad
      }
    }
    return result;
  } finally {
    running = false;
  }
}
```

Why it's safe:

- **Never twice:** each period has a unique key (`subscriptionId + periodStart`). Run billing 10 times and you still get one invoice per period.
- **Never skipped:** the `while` loop catches up on every missed period. Each period is its own transaction, so a crash halfway leaves the data consistent, and the next run continues from where it stopped.
- **Money:** all calculations are done by Person 3's `computeInvoiceAmounts` in paise, so there are no rounding errors.

`server/src/services/billingCron.js`

```js
import cron from 'node-cron';
import { runBilling } from './billing.js';

export function startBillingCron() {
  // every day at 01:00 server time
  cron.schedule('0 1 * * *', async () => {
    try {
      console.log('[billing cron]', await runBilling());
    } catch (e) {
      console.error('[billing cron] failed:', e.message);
    }
  });
}
```

`server/src/routes/billing.js`

```js
import { Router } from 'express';
import { requireAuth, requireRole } from '../middleware/auth.js';
import { runBilling } from '../services/billing.js';

const router = Router();

router.post('/run', requireAuth, requireRole('ADMIN', 'STAFF'), async (req, res) => {
  try {
    res.json(await runBilling({ autoConfirm: !!req.body?.autoConfirm }));
  } catch (e) {
    res.status(409).json({ error: e.message });
  }
});

export default router;
```

4. Invoices: `server/src/routes/invoices.js`

```js
```

```
import { Router } from 'express';
import { prisma } from '../index.js';
import { requireAuth, requireRole } from '../middleware/auth.js';
import { invoiceStatusSchema } from '../schemas/billing.js';

const router = Router();
const staffOrAdmin = [requireAuth, requireRole('ADMIN', 'STAFF')];
const n = (v) => Number(v ?? 0);

const withDerived = (inv) => {
  const outstanding = Math.round((n(inv.total) - n(inv.amountPaid)) * 100) / 100;
  return { ...inv, outstanding, isOverdue: inv.status === 'CONFIRMED' && inv.dueDate <
};

// LIST: filters + pagination. ?status= &overdue=true &customerId= &search= &from= &to
router.get('/', staffOrAdmin, async (req, res) => {
  const page = Number(req.query.page) || 1;
  const limit = Math.min(Number(req.query.limit) || 10, 100);
  const { status, overdue, customerId, search, from, to } = req.query;

  const where = {
  ...(status && { status }),
  ...(overdue === 'true' && { status: 'CONFIRMED', dueDate: { lt: new Date() } }),
  ...(customerId && { customerId }),
  ...((from || to) && { issueDate: { ...(from && { gte: new Date(from) }), ...(to &&
  ...(search && {
    OR: [
      { number: { contains: search, mode: 'insensitive' } },
      { customer: { name: { contains: search, mode: 'insensitive' } } },
      { subscription: { number: { contains: search, mode: 'insensitive' } } },
    ],
  }),
};

const [items, total] = await Promise.all([
  prisma.invoice.findMany({
    where,
    include: { customer: { select: { id: true, name: true, email: true } }, subscrip
    orderBy: { issueDate: 'desc' },
    skip: (page - 1) * limit,
    take: limit,
  }),
  prisma.invoice.count({ where }),
]);
res.json({ items: items.map(withDerived), total, page, pages: Math.ceil(total / limi
});

router.get('/:id', staffOrAdmin, async (req, res) => {
  const inv = await prisma.invoice.findUnique({
    where: { id: req.params.id },
    include: {
      customer: { select: { id: true, name: true, email: true } },
      subscription: { select: { number: true } },
      lines: true,
      payments: { orderBy: { date: 'desc' } },
    },
  });
  if (!inv) return res.status(404).json({ error: 'Invoice not found' });
  res.json(withDerived(inv));
});

// CONFIRM / CANCEL
const FLOW = { DRAFT: ['CONFIRMED', 'CANCELLED'], CONFIRMED: ['CANCELLED'], PAID: [],

router.patch('/:id/status', staffOrAdmin, async (req, res) => {
  const parsed = invoiceStatusSchema.safeParse(req.body);
```

```
  if (!parsed.success) return res.status(400).json({ error: parsed.error.issues[0].mes
  const inv = await prisma.invoice.findUnique({ where: { id: req.params.id } });
  if (!inv) return res.status(404).json({ error: 'Invoice not found' });

  const { status } = parsed.data;
  if (!FLOW[inv.status].includes(status)) return res.status(400).json({ error: `Cannot
  if (status === 'CANCELLED' && n(inv.amountPaid) > 0) return res.status(400).json({ e

  res.json(await prisma.invoice.update({ where: { id: inv.id }, data: { status } }));
});

// SEND (no real email needed: log it and stamp sentAt. If you have time, use nodemail
router.post('/:id/send', staffOrAdmin, async (req, res) => {
  const inv = await prisma.invoice.findUnique({ where: { id: req.params.id }, include:
  if (!inv) return res.status(404).json({ error: 'Invoice not found' });
  if (!['CONFIRMED', 'PAID'].includes(inv.status)) return res.status(400).json({ error
  console.log(`[email] invoice ${inv.number} sent to ${inv.customer.email}`);
  res.json(await prisma.invoice.update({ where: { id: inv.id }, data: { sentAt: new Da
});

export default router;
```

Print: no backend needed. The detail page calls `window.print()`, and Tailwind's `print:hidden` class hides buttons, the sidebar and the header (see section 6).

5. Payments: `server/src/routes/payments.js`

```
js
```

```
import { Router } from 'express';
import { prisma } from '../index.js';
import { requireAuth, requireRole } from '../middleware/auth.js';
import { paymentSchema } from '../schemas/billing.js';

const router = Router();
const staffOrAdmin = [requireAuth, requireRole('ADMIN', 'STAFF')];
const toPaise = (v) => Math.round(Number(v) * 100);

router.get('/', staffOrAdmin, async (req, res) => {
  const page = Number(req.query.page) || 1;
  const limit = Math.min(Number(req.query.limit) || 10, 100);
  const { method, invoiceId, from, to } = req.query;
  const where = {
    ...(method && { method }),
    ...(invoiceId && { invoiceId }),
    ...((from || to) && { date: { ...(from && { gte: new Date(from) }), ...(to && { lt
  };
  const [items, total] = await Promise.all([
    prisma.payment.findMany({
      where,
      include: { invoice: { select: { id: true, number: true, customer: { select: { na
      orderBy: { date: 'desc' },
      skip: (page - 1) * limit,
      take: limit,
    }),
    prisma.payment.count({ where }),
  ]);
  res.json({ items, total, page, pages: Math.ceil(total / limit) });
});

// RECORD A PAYMENT against an invoice
router.post('/', staffOrAdmin, async (req, res) => {
  const parsed = paymentSchema.safeParse(req.body);
  if (!parsed.success) return res.status(400).json({ error: parsed.error.issues[0].mes
  const { invoiceId, method, amount, date, note } = parsed.data;

  try {
    const payment = await prisma.$transaction(async (tx) => {
      const inv = await tx.invoice.findUnique({ where: { id: invoiceId } });
      if (!inv) throw new Error('Invoice not found');
      if (inv.status !== 'CONFIRMED') throw new Error('Only confirmed, unpaid invoices

      const outstanding = toPaise(inv.total) - toPaise(inv.amountPaid);
      if (toPaise(amount) > outstanding) throw new Error(`Amount is more than the outs

      const paid = toPaise(inv.amountPaid) + toPaise(amount);
      await tx.invoice.update({
        where: { id: invoiceId },
        data: { amountPaid: paid / 100, status: paid >= toPaise(inv.total) ? 'PAID' :
      });
      return tx.payment.create({ data: { invoiceId, method, amount, date, note } });
    });
    res.status(201).json(payment);
  } catch (e) {
    res.status(400).json({ error: e.message });
  }
});

export default router;
```

6. Dashboard API: `server/src/routes/dashboard.js`

```
js
```

```
import { Router } from 'express';
import { prisma } from '../index.js';
import { requireAuth, requireRole } from '../middleware/auth.js';

const router = Router();
const n = (v) => Number(v ?? 0);

router.get('/summary', requireAuth, requireRole('ADMIN', 'STAFF'), async (req, res) =>
  const now = new Date();
  const from = req.query.from ? new Date(req.query.from) : new Date(Date.UTC(now.getUT
  const to = req.query.to ? new Date(req.query.to) : new Date(now);
  to.setUTCHours(23, 59, 59, 999);

  const [active, byStatus, revenue, invoiced, open, overdue, payM, invM] = await Promi
    prisma.subscription.count({ where: { status: 'ACTIVE' } }),
    prisma.invoice.groupBy({ by: ['status'], where: { issueDate: { gte: from, lte: to
    prisma.payment.aggregate({ where: { date: { gte: from, lte: to } }, _sum: { amount
    prisma.invoice.aggregate({ where: { status: { not: 'CANCELLED' }, issueDate: { gte
    prisma.invoice.aggregate({ where: { status: 'CONFIRMED' }, _sum: { total: true, am
    prisma.invoice.aggregate({ where: { status: 'CONFIRMED', dueDate: { lt: now } }, _
    prisma.$queryRaw`SELECT to_char(date_trunc('month', "date"), 'YYYY-MM') AS month,
                     FROM "Payment" WHERE "date" >= ${from} AND "date" <= ${to} GROUP
    prisma.$queryRaw`SELECT to_char(date_trunc('month', "issueDate"), 'YYYY-MM') AS mo
                     FROM "Invoice" WHERE "status" <> 'CANCELLED' AND "issueDate" >= $
  ]);

  // merge the two monthly series into one array for the chart
  const months = {};
  invM.forEach((r) => (months[r.month] = { month: r.month, invoiced: r.total, collecte
  payM.forEach((r) => ((months[r.month] ||= { month: r.month, invoiced: 0, collected:

  res.json({
    activeSubscriptions: active,
    revenueCollected: n(revenue._sum.amount),
    totalInvoiced: n(invoiced._sum.total),
    outstanding: { amount: n(open._sum.total) - n(open._sum.amountPaid), count: open._
    overdue: { amount: n(overdue._sum.total) - n(overdue._sum.amountPaid), count: over
    byStatus: byStatus.map((s) => ({ status: s.status, count: s._count._all })),
    monthly: Object.values(months).sort((a, b) => a.month.localeCompare(b.month)),
  });
});

export default router;
```

Mount everything in `server/src/index.js` (add only these)

```
js

import billingRoutes from './routes/billing.js';
import invoiceRoutes from './routes/invoices.js';
import paymentRoutes from './routes/payments.js';
import dashboardRoutes from './routes/dashboard.js';
import { startBillingCron } from './services/billingCron.js';

app.use('/api/billing', billingRoutes);
app.use('/api/invoices', invoiceRoutes);
app.use('/api/payments', paymentRoutes);
app.use('/api/dashboard', dashboardRoutes);

// after app.listen(...)
startBillingCron();
```

If Person 1's auth isn't merged yet, use the temporary stub for `requireAuth` and `requireRole` (they just call `next()`) and don't commit it. The export names must be exactly `requireAuth` and `requireRole('ADMIN', 'STAFF')`.

7. Frontend

`Dashboard.jsx` (the showpiece, so make it look good)

jsx

```
import { useEffect, useState } from 'react';
import { ResponsiveContainer, BarChart, Bar, XAxis, YAxis, Tooltip, Legend, CartesianG
import { Repeat, IndianRupee, Clock, AlertTriangle } from 'lucide-react';
import { api } from '../../api/client';
import Table from '../../components/Table';
import StatusBadge from '../../components/StatusBadge';
import { inputClass } from '../../components/FormField';

const money = (v) => `₹${Number(v || 0).toLocaleString('en-IN', { maximumFractionDigit
const COLORS = { DRAFT: '#94a3b8', CONFIRMED: '#f59e0b', PAID: '#22c55e', CANCELLED: '

function Kpi({ icon: Icon, label, value, sub, tone }) {
  return (
    <div className="rounded-xl border bg-white p-4">
      <div className={`mb-2 inline-flex rounded-lg p-2 ${tone}`}><Icon size={18} /></d
      <p className="text-sm text-slate-500">{label}</p>
      <p className="text-2xl font-semibold">{value}</p>
      {sub && <p className="text-xs text-slate-400">{sub}</p>}
    </div>
  );
}

export default function Dashboard() {
  const [range, setRange] = useState({ from: '', to: '' });
  const [data, setData] = useState(null);
  const [overdue, setOverdue] = useState([]);
  const [error, setError] = useState('');

  const load = () => {
    const q = new URLSearchParams(Object.entries(range).filter(([, v]) => v)).toString
    api(`/dashboard/summary?${q}`).then(setData).catch((e) => setError(e.message));
    api('/invoices?overdue=true&limit=5').then((r) => setOverdue(r.items)).catch(() =>
  };

  useEffect(() => { load(); const t = setInterval(load, 30000); return () => clearInte

  if (error) return <p className="text-red-600">{error}</p>;
  if (!data) return <p className="text-slate-400">Loading...</p>;

  return (
    <div className="space-y-6">
      <div className="flex flex-wrap items-center justify-between gap-3">
        <h2 className="text-2xl font-semibold">Finance dashboard</h2>
        <div className="flex items-center gap-2 text-sm">
          <input type="date" className={inputClass} value={range.from} onChange={(e) =
          <span>to</span>
          <input type="date" className={inputClass} value={range.to} onChange={(e) =>
        </div>
      </div>

      <div className="grid grid-cols-1 gap-4 sm:grid-cols-2 lg:grid-cols-4">
        <Kpi icon={Repeat} tone="bg-indigo-100 text-indigo-600" label="Active subscrip
        <Kpi icon={IndianRupee} tone="bg-green-100 text-green-600" label="Revenue coll
        <Kpi icon={Clock} tone="bg-amber-100 text-amber-600" label="Outstanding" value
        <Kpi icon={AlertTriangle} tone="bg-red-100 text-red-600" label="Overdue" value
      </div>

      <div className="grid gap-4 lg:grid-cols-3">
        <div className="rounded-xl border bg-white p-4 lg:col-span-2">
          <h3 className="mb-3 font-medium">Invoiced vs collected, by month</h3>
          <ResponsiveContainer width="100%" height={280}>
            <BarChart data={data.monthly}>
              <CartesianGrid strokeDasharray="3 3" />
              <XAxis dataKey="month" />
              <YAxis tickFormatter={(v) => `${v / 1000}k`} />
              <Tooltip formatter={(v) => money(v)} />
```

```
            <Legend />
            <Bar dataKey="invoiced" name="Invoiced" fill="#6366f1" />
            <Bar dataKey="collected" name="Collected" fill="#22c55e" />
          </BarChart>
        </ResponsiveContainer>
      </div>
      <div className="rounded-xl border bg-white p-4">
        <h3 className="mb-3 font-medium">Invoices by status</h3>
        <ResponsiveContainer width="100%" height={280}>
          <PieChart>
            <Pie data={data.byStatus} dataKey="count" nameKey="status" outerRadius={
              {data.byStatus.map((s) => <Cell key={s.status} fill={COLORS[s.status]}
            </Pie>
            <Tooltip /><Legend />
          </PieChart>
        </ResponsiveContainer>
      </div>
    </div>

    <div>
      <h3 className="mb-2 font-medium">Overdue invoices (chase these first)</h3>
      <Table
        rows={overdue} page={1} pages={1} empty="Nothing overdue 🎉"
        columns={[
          { label: 'Invoice', render: (r) => <a className="text-indigo-600" href={`/
          { label: 'Customer', render: (r) => r.customer.name },
          { label: 'Due', render: (r) => r.dueDate.slice(0, 10) },
          { label: 'Outstanding', render: (r) => money(r.outstanding) },
          { label: 'Status', render: () => <StatusBadge status="OVERDUE" /> },
        ]}
      />
    </div>
  );
}
```

`Invoices.jsx` (list at `/invoices`)

- Top bar: search box (invoice number, customer, subscription), a status `<select>` (All, Draft, Confirmed, Paid, Cancelled), an "**Overdue only**" checkbox, and from and to date inputs.
- **Run Billing button** (top right, indigo):

jsx

```
const runBilling = async () => {
  if (!confirm('Generate invoices for all due subscriptions?')) return;
  setBusy(true);
  try {
    const r = await api('/billing/run', { method: 'POST', body: {} });
    setMsg(`Created ${r.invoicesCreated} invoices, skipped ${r.skipped} already bill
    load();
  } catch (e) { setMsg(e.message); }
  setBusy(false);
};
```

Disable the button while `busy` and show the result in a green or red banner. Clicking it twice and seeing "Created 0, skipped N" is the best demo moment.

- Table columns: Number, Customer, Subscription, Issue date, Due date, Total, Paid, Outstanding, Status (`StatusBadge`, but show `OVERDUE` instead when `r.isOverdue`), and a View link to `/invoices/${id}`.
- Use the shared `Table` with pagination, and reset to page 1 whenever a filter changes.

`InvoiceDetail.jsx` (at `/invoices/:id`)

- Header: invoice number, `StatusBadge`, customer name and email, subscription number, issue date, due date, "Sent on …" if `sentAt` is set.
- **Action buttons** (wrap them in a div with `print:hidden`) depending on status:
  - DRAFT: **Confirm** (`PATCH /invoices/:id/status` with `{status:'CONFIRMED'}`) and

  - Cancel
  - CONFIRMED: **Record payment**, **Send**, **Print**, and **Cancel** (disable it if `amountPaid > 0`)
  - PAID: **Send** and **Print**
  - Show the server's error message if any call fails.
- Lines table: Description, Qty, Unit price, Discount, Tax %, Tax, Line total.
- Totals box on the right: Subtotal, Discount, Tax, **Total**, Paid, **Outstanding**.
- Payments history table below: date, method, amount, note.
- **Record payment modal** (`Modal` + `FormField`):
  - method `<select>`
  - amount (`type="number" step="0.01" min="0.01" max={outstanding}`, default filled with the outstanding amount)
  - date (`type="date"`, `max` = today, default today)
  - note (textarea, max 500)
  - Client check: amount greater than 0 and not above the outstanding. On submit, `POST /payments` with `{ invoiceId, method, amount, date, note }`, then reload.
- **Print:** the Print button runs `window.print()`.

`Payments.jsx` (at `/payments`)

- Filters: method `<select>`, from and to dates.
- Columns: Date, Invoice (link to `/invoices/:id`), Customer, Method, Amount, Note. Paginated.
- A "Record payment" button here is optional. Recording from the invoice page is enough.

`Layout.jsx` (one tiny edit)

Add `print:hidden` to the classes of the `<aside>` and the `<header>`, so printed invoices don't include the sidebar. Tell Person 1.

`App.jsx`: replace only your routes

jsx

```jsx
import Dashboard from './pages/dashboard/Dashboard';
import Invoices from './pages/staff/Invoices';
import InvoiceDetail from './pages/staff/InvoiceDetail';
import Payments from './pages/staff/Payments';

<Route path="/dashboard" element={<Dashboard />} />
<Route path="/invoices" element={<Invoices />} />
<Route path="/invoices/:id" element={<InvoiceDetail />} />
<Route path="/payments" element={<Payments />} />
```

8. The seed script: `server/prisma/seed.js`

This fills the app with about 200 subscriptions and about 1000 invoices, using **the real billing engine**. That means the seed also tests your billing. Run it with `npm run seed` from `server/`. It wipes the database first, so tell the group before anyone runs it.

js

```
// Setting PORT=0 stops importing index.js from clashing with a running server
process.env.PORT = '0';
const { prisma } = await import('../src/index.js');
const { runBilling } = await import('../src/services/billing.js');
const bcrypt = (await import('bcryptjs')).default;

const DAY = 86400000;
const now = new Date();
const daysAgo = (d) => new Date(now.getTime() - d * DAY);
const daysAhead = (d) => new Date(now.getTime() + d * DAY);
const rand = (a, b) => Math.floor(Math.random() * (b - a + 1)) + a;
const pick = (arr) => arr[rand(0, arr.length - 1)];

console.log('Wiping old data...');
for (const m of ['payment', 'invoiceLine', 'invoice', 'subscriptionLine', 'subscriptio
  'quotationTemplate', 'discount', 'tax', 'productVariant', 'product', 'plan', 'user']
  await prisma[m].deleteMany();
}

console.log('Users...');
const hash = await bcrypt.hash('Password@123', 8);
await prisma.user.create({ data: { email: 'admin@brightloop.com', name: 'Admin', passw
await prisma.user.create({ data: { email: 'staff@brightloop.com', name: 'Staff Member'
const first = ['Aarav', 'Diya', 'Kabir', 'Meera', 'Rohan', 'Isha', 'Vikram', 'Anaya',
const last = ['Nair', 'Menon', 'Iyer', 'Kumar', 'Shah', 'Pillar', 'Das', 'Rao'];
const customers = [];
for (let i = 1; i <= 40; i++) {
  customers.push(await prisma.user.create({
    data: { email: `customer${i}@example.com`, name: `${pick(first)} ${pick(last)} Pvt
  }));
}

console.log('Taxes, products, plans, discounts...');
const taxes = await Promise.all([5, 12, 18].map((p) => prisma.tax.create({ data: { nam

const productData = [
  ['Brightloop CRM', 'Software', 1500, 600], ['Brightloop Invoicing', 'Software', 1200
  ['Brightloop HR Suite', 'Software', 2500, 900], ['Brightloop Support Desk', 'Add-on'
  ['Brightloop Analytics', 'Add-on', 1800, 700],
];
const products = [];
for (const [name, type, salesPrice, costPrice] of productData) {
  const p = await prisma.product.create({ data: { name, type, salesPrice, costPrice, n
  await prisma.productVariant.createMany({ data: [
    { productId: p.id, attribute: 'Brand', value: 'Odoo', extraPrice: 560 },
    { productId: p.id, attribute: 'Brand', value: 'Zoho', extraPrice: 320 },
  ] });
  products.push({ ...p, variants: await prisma.productVariant.findMany({ where: { prod
}

const mkPlan = (name, price, billingPeriod) => prisma.plan.create({ data: {
  name, price, billingPeriod, minQuantity: 1, availableFrom: daysAgo(400), availableTo
  autoClose: true, allowClose: true, allowPause: true, allowRenew: true,
} });
const monthly = await mkPlan('Monthly Standard', 999, 'MONTHLY');
const yearly = await mkPlan('Yearly Business', 9999, 'YEARLY');
const weekly = await mkPlan('Weekly Starter', 299, 'WEEKLY');
await mkPlan('Daily Trial', 49, 'DAILY');

await prisma.discount.create({ data: {
  name: '10% off CRM', type: 'PERCENT', value: 10, appliesTo: 'PRODUCT', productId: pr
  startDate: daysAgo(120), endDate: daysAhead(60),
} });
await prisma.discount.create({ data: {
  name: '₹500 off big accounts', type: 'FIXED', value: 500, appliesTo: 'SUBSCRIPTION',
  startDate: daysAgo(120), endDate: daysAhead(60), usageLimit: 40, // runs out on its
```

```
} });

console.log('Subscriptions...');
let seq = 0;
for (let i = 0; i < 200; i++) {
  const r = Math.random();
  const [plan, startAgo] = r < 0.7 ? [monthly, rand(1, 170)] : r < 0.9 ? [yearly, rand
  const start = daysAgo(startAgo);
  const s = Math.random();
  const status = s < 0.85 ? 'ACTIVE' : s < 0.92 ? 'CLOSED' : pick(['DRAFT', 'QUOTATION

  const lines = Array.from({ length: rand(1, 2) }, () => {
    const p = pick(products);
    const v = Math.random() < 0.5 ? pick(p.variants) : null;
    const quantity = rand(1, 10);
    const unitPrice = Number(p.salesPrice) + Number(v?.extraPrice || 0);
    return { productId: p.id, variantId: v?.id ?? null, quantity, unitPrice, taxId: pi
  });

  seq++;
  await prisma.subscription.create({ data: {
    number: `SUB-${String(seq).padStart(5, '0')}`,
    customerId: pick(customers).id, planId: plan.id,
    startDate: start, expiryDate: new Date(start.getTime() + 365 * DAY),
    paymentTerms: pick(['IMMEDIATE', 'NET_15', 'NET_30', 'NET_30', 'NET_60']),
    status, nextBillingDate: status === 'ACTIVE' ? start : null,
    lines: { create: lines },
  } });
}

console.log('Running the real billing engine (can take 30-90 seconds)...');
console.log(await runBilling({ asOf: now }));
console.log('Run it again: nothing should be created.');
console.log(await runBilling({ asOf: now }));

console.log('Confirming invoices and recording payments...');
await prisma.invoice.updateMany({ where: { status: 'DRAFT', number: { not: { endsWith:
const confirmed = await prisma.invoice.findMany({ where: { status: 'CONFIRMED' } });
const methods = ['CASH', 'CARD', 'UPI', 'BANK_TRANSFER', 'CHEQUE'];

for (const inv of confirmed) {
  const roll = Math.random();
  if (roll > 0.8) continue; // 20% stay unpaid, the older ones become overdue
  const total = Number(inv.total);
  const amount = roll < 0.7 ? total : Math.round(total * 0.5 * 100) / 100; // 70% full
  const date = new Date(Math.min(now.getTime(), inv.issueDate.getTime() + rand(0, 20)
  await prisma.payment.create({ data: { invoiceId: inv.id, method: pick(methods), amou
  await prisma.invoice.update({ where: { id: inv.id }, data: { amountPaid: amount, sta
}

console.log('Done. Login: admin@brightloop.com / staff@brightloop.com / customer1@exam
await prisma.$disconnect();
process.exit(0);
```

Make sure `server/package.json` has `"seed": "node prisma/seed.js"` (the foundation already does).

If you need to tweak it: the demo works best with roughly 15 to 20% of invoices overdue and some partial payments, so the dashboard has something to chase.

9. Contracts with the others (don't change without telling them)

Person 1 (customer portal) will use:

- Customer invoices: `prisma.invoice.findMany({ where: { customerId: req.user.id, status: { in: ['CONFIRMED', 'PAID'] } }, include: { lines: true, payments: true }, orderBy: { issueDate: 'desc' } })`. Customers must never see draft or cancelled invoices, and never another customer's.
- Seed logins all use the password `Password@123`.

Person 3 provides:

- `computeInvoiceAmounts(subscription, asOf, tx)` returns `{ lines, subtotal, discountTotal, taxTotal, total, appliedDiscountIds }`, and `markDiscountsUsed(ids, tx)`. If they change these, billing breaks, so ask them to keep the names stable.
- Active subscriptions have `nextBillingDate` set (they set it to `startDate` on activation). Billing moves it forward.

Everyone can reuse from you: `StatusBadge` accepts `PAID`, `CANCELLED` and `OVERDUE`.

10. Test it properly (this is what the judges will poke)

1. Run `npm run seed`. The second `runBilling` in the log must say `invoicesCreated: 0`. That proves nothing is billed twice.
2. In the app, click **Run Billing** twice. The second time: "Created 0".
3. **Missed periods:** in the database or Prisma Studio (`npx prisma studio`), set one ACTIVE monthly subscription's `nextBillingDate` back by 3 months and delete its invoices, then run billing. You should get 3 invoices.
4. **Maths check:** a line with qty 10, ₹1000 unit price, 18% tax and a 10% product discount must give an invoice total of **₹10,620**.
5. Pay an invoice partially: it stays CONFIRMED with the right outstanding. Pay the rest: it flips to PAID. Try paying more than the outstanding: it must be rejected.
6. Try recording a payment against a DRAFT invoice: it must be rejected.
7. Dashboard: after paying something, the numbers update within 30 seconds (or on refresh).
8. Open `/dashboard` and `/invoices` with the seeded 1000+ invoices. Pages should respond in under 2 seconds. If a list is slow, check that the indexes were migrated.

11. Rules

- Money is **never Float:** `Decimal` in the database, paise for comparisons, `Number()` in the UI.
- Billing must stay **one transaction per subscription period.** Don't wrap the whole run in a single transaction.
- All lists need pagination and filters. Never `findMany` without `take`.
- Writes (confirm, cancel, send, payments, run billing) are for admin and staff only, and customers never reach these routes.
- Don't edit Person 3's `pricing.js` yourself. If it has a bug, tell them.

12. Checklist

- [ ] Schema migrated with `Invoice`, `InvoiceLine` and `Payment` (plus the 2 back-relations)
- [ ] Run Billing creates invoices with customer, lines, tax and totals filled in
- [ ] Second run creates nothing, and missed periods are caught up
- [ ] Invoice confirm, cancel, send and print all work
- [ ] Payments work, with PAID/partial/outstanding handled
- [ ] Overdue list works and shows on the dashboard
- [ ] Dashboard: 4 KPIs, a bar chart, a pie chart, date filter, and the overdue table
- [ ] Seed script runs end to end, and the team has been told the logins
- [ ] Phone-width screens look fine (tables scroll sideways, KPI cards stack)

13. Git

```bash
git add .
git commit -m "feat: billing engine, invoices, payments, dashboard, seed"
git push origin feature/p4-billing
```

Commit small and often (schema, billing service, invoice routes, payments, dashboard API, each page, seed). Open a PR as soon as the **billing service and seed** work, since everyone else needs the seeded data to test with. Don't save them for the last hour.

for person 1

Checking the mailer and the user management routes ▸

Here's everything Person 1 needs: **Auth, roles, staff/customer management and the customer portal.** You're also the person the other three are waiting on. Send it as is.

0. Before you start

```bash
git pull origin main
git checkout -b feature/p1-auth
```

Your first 45 minutes unblock everyone else. Do these in order and push as soon as they're done:

1. `middleware/auth.js` (the others have a temporary stub of it, and yours replaces it)
2. The `App.jsx` restructure with `ProtectedRoute`
3. `createAdmin.js`, so someone can actually log in
4. Auth routes (register, login, me)

Then tell the group: "Auth is on main. Pull, delete your temporary auth stub, and run `npm run create-admin`." After that, finish reset password, staff/customer pages and the portal.

The User table already has everything you need (`resetToken`, `resetTokenExpiry`, `role`), so you probably don't need a migration.

Files you own (create these):

```
server/src/middleware/auth.js
server/src/schemas/auth.js
server/src/services/mailer.js
server/src/routes/auth.js
server/src/routes/users.js
server/src/routes/me.js
server/prisma/createAdmin.js
client/src/utils/validators.js
client/src/routes/ProtectedRoute.jsx
client/src/components/PasswordChecklist.jsx
client/src/pages/auth/Login.jsx
client/src/pages/auth/Register.jsx
client/src/pages/auth/ForgotPassword.jsx
client/src/pages/auth/ResetPassword.jsx
client/src/pages/admin/Staff.jsx
client/src/pages/staff/Customers.jsx
client/src/pages/portal/MySubscriptions.jsx
client/src/pages/portal/MyInvoices.jsx
client/src/pages/portal/MyInvoiceDetail.jsx
```

Shared files you edit:

```
server/src/index.js        (mount 3 routes, add an error handler)
server/.env.example        (add 4 variables)
client/src/App.jsx         (restructure, see section 5)
client/src/api/client.js   (handle expired sessions)
client/src/components/Layout.jsx (small edits)
```

You also own the foundation, so you're the person who answers "it doesn't run" messages.

1. Environment variables

Add to `server/.env.example` and your own `.env`:

```
JWT_SECRET="any-long-random-string-here"
CLIENT_URL="http://localhost:5173"
# Optional: real email via Ethereal (free test inbox at ethereal.email). Leave empty
to just print reset links in the server console.
SMTP_HOST=""
SMTP_PORT=587
SMTP_USER=""
SMTP_PASS=""
MAIL_FROM="Brightloop <no-reply@brightloop.test>"
```

2. Backend

`server/src/schemas/auth.js`

The password rule from the brief: **more than 8 characters**, with an uppercase letter, a lowercase

letter and a special character.

```js
import { z } from 'zod';

export const passwordSchema = z
  .string()
  .min(9, 'Password must be more than 8 characters')
  .regex(/[A-Z]/, 'Password needs an uppercase letter')
  .regex(/[a-z]/, 'Password needs a lowercase letter')
  .regex(/[^A-Za-z0-9]/, 'Password needs a special character');

const email = z.string().trim().toLowerCase().email('Enter a valid email address');
const name = z.string().trim().min(2, 'Name must be at least 2 characters').max(80);

export const registerSchema = z.object({ name, email, password: passwordSchema });
export const createUserSchema = registerSchema; // staff/customer created by admin or
export const loginSchema = z.object({ email, password: z.string().min(1, 'Password is
export const forgotSchema = z.object({ email });
export const resetSchema = z.object({ token: z.string().min(10, 'Invalid link'), passw
```

`server/src/middleware/auth.js` **(push this first)**

The other three import exactly these names, so don't rename them.

```js
import jwt from 'jsonwebtoken';
import { prisma } from '../index.js';

export async function requireAuth(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ error: 'Please log in' });
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    const user = await prisma.user.findUnique({
      where: { id: payload.id },
      select: { id: true, name: true, email: true, role: true },
    });
    if (!user) return res.status(401).json({ error: 'Account no longer exists' });
    req.user = user; // { id, name, email, role } for every route after this
    next();
  } catch {
    res.status(401).json({ error: 'Session expired, please log in again' });
  }
}

export const requireRole = (...roles) => (req, res, next) =>
  roles.includes(req.user?.role)
    ? next()
    : res.status(403).json({ error: 'You do not have access to this' });
```

It reads the role from the **database** on every request, not just from the token. So if a role changes, it takes effect immediately.

`server/src/services/mailer.js`

```js
```

```
import nodemailer from 'nodemailer';

const transporter = process.env.SMTP_HOST
  ? nodemailer.createTransport({
      host: process.env.SMTP_HOST,
      port: Number(process.env.SMTP_PORT || 587),
      auth: { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS },
    })
  : null;

export async function sendResetEmail(to, link) {
  // always print it, so the demo works with no internet and no SMTP
  console.log(`\n[password reset] ${to}\n${link}\n`);
  if (!transporter) return;
  try {
    await transporter.sendMail({
      from: process.env.MAIL_FROM,
      to,
      subject: 'Reset your Brightloop password',
      text: `Use this link to reset your password (valid for 1 hour):\n${link}`,
      html: `<p>Use this link to reset your password (valid for 1 hour):</p><p><a href
    });
  } catch (e) {
    console.error('[mail failed]', e.message); // never break the request because emai
  }
}
```

`server/src/routes/auth.js`

```
js
```

```
import { Router } from 'express';
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import crypto from 'crypto';
import { prisma } from '../index.js';
import { requireAuth } from '../middleware/auth.js';
import { registerSchema, loginSchema, forgotSchema, resetSchema } from '../schemas/aut
import { sendResetEmail } from '../services/mailer.js';

const router = Router();
const sign = (u) => jwt.sign({ id: u.id, role: u.role }, process.env.JWT_SECRET, { exp
const publicUser = (u) => ({ id: u.id, name: u.name, email: u.email, role: u.role });
const hashToken = (t) => crypto.createHash('sha256').update(t).digest('hex');
const firstError = (parsed) => parsed.error.issues[0].message;

// SIGN UP: always creates a CUSTOMER, whatever the body says
router.post('/register', async (req, res) => {
  const parsed = registerSchema.safeParse(req.body);
  if (!parsed.success) return res.status(400).json({ error: firstError(parsed) });
  const { name, email, password } = parsed.data;
  try {
    const user = await prisma.user.create({
      data: { name, email, passwordHash: await bcrypt.hash(password, 10), role: 'CUSTO
    });
    res.status(201).json({ token: sign(user), user: publicUser(user) });
  } catch (e) {
    if (e.code === 'P2002') return res.status(409).json({ error: 'An account with this
    throw e;
  }
});

router.post('/login', async (req, res) => {
  const parsed = loginSchema.safeParse(req.body);
  if (!parsed.success) return res.status(400).json({ error: firstError(parsed) });
  const { email, password } = parsed.data;
  const user = await prisma.user.findUnique({ where: { email } });
  const ok = user && (await bcrypt.compare(password, user.passwordHash));
  if (!ok) return res.status(401).json({ error: 'Invalid email or password' }); // sam
  res.json({ token: sign(user), user: publicUser(user) });
});

router.get('/me', requireAuth, (req, res) => res.json(req.user));

// FORGOT PASSWORD: always answers the same, so nobody can discover which emails exist
router.post('/forgot-password', async (req, res) => {
  const parsed = forgotSchema.safeParse(req.body);
  if (!parsed.success) return res.status(400).json({ error: firstError(parsed) });
  const user = await prisma.user.findUnique({ where: { email: parsed.data.email } });
  if (user) {
    const token = crypto.randomBytes(32).toString('hex');
    await prisma.user.update({
      where: { id: user.id },
      data: { resetToken: hashToken(token), resetTokenExpiry: new Date(Date.now() + 60
    });
    await sendResetEmail(user.email, `${process.env.CLIENT_URL}/reset-password?token=$
  }
  res.json({ message: 'If that email is registered, a reset link has been sent.' });
});

router.post('/reset-password', async (req, res) => {
  const parsed = resetSchema.safeParse(req.body);
  if (!parsed.success) return res.status(400).json({ error: firstError(parsed) });
  const { token, password } = parsed.data;
  const user = await prisma.user.findFirst({
    where: { resetToken: hashToken(token), resetTokenExpiry: { gt: new Date() } },
  });
```

```
  if (!user) return res.status(400).json({ error: 'This reset link is invalid or has e
  await prisma.user.update({
    where: { id: user.id },
    data: { passwordHash: await bcrypt.hash(password, 10), resetToken: null, resetToke
  });
  res.json({ message: 'Password updated. You can log in now.' });
});

export default router;
```

Why the tokens are hashed: the database stores only a SHA-256 hash of the reset token, like a password. If someone sees the table, they can't use the links.

`server/src/routes/users.js`

```
js
```

```
import { Router } from 'express';
import bcrypt from 'bcryptjs';
import { prisma } from '../index.js';
import { requireAuth, requireRole } from '../middleware/auth.js';
import { createUserSchema } from '../schemas/auth.js';

const router = Router();
const staffOrAdmin = [requireAuth, requireRole('ADMIN', 'STAFF')];
const adminOnly = [requireAuth, requireRole('ADMIN')];
const select = { id: true, name: true, email: true, role: true, createdAt: true };

// LIST: GET /api/users?role=CUSTOMER|STAFF&search=&page=&limit=
router.get('/', staffOrAdmin, async (req, res) => {
  const role = (req.query.role || 'CUSTOMER').toString();
  if (!['CUSTOMER', 'STAFF'].includes(role)) return res.status(400).json({ error: 'Inv
  if (role === 'STAFF' && req.user.role !== 'ADMIN') return res.status(403).json({ err

  const page = Number(req.query.page) || 1;
  const limit = Math.min(Number(req.query.limit) || 10, 200);
  const search = req.query.search?.toString().trim();
  const where = {
    role,
    ...(search && { OR: [{ name: { contains: search, mode: 'insensitive' } }, { email:
  };
  const [items, total] = await Promise.all([
    prisma.user.findMany({ where, select, orderBy: { createdAt: 'desc' }, skip: (page
    prisma.user.count({ where }),
  ]);
  res.json({ items, total, page, pages: Math.ceil(total / limit) });
});

async function createWithRole(role, req, res) {
  const parsed = createUserSchema.safeParse(req.body);
  if (!parsed.success) return res.status(400).json({ error: parsed.error.issues[0].mes
  const { name, email, password } = parsed.data;
  try {
    const user = await prisma.user.create({
      data: { name, email, passwordHash: await bcrypt.hash(password, 10), role },
      select,
    });
    res.status(201).json(user);
  } catch (e) {
    if (e.code === 'P2002') return res.status(409).json({ error: 'An account with this
    throw e;
  }
}

// staff can add customers (day-to-day work), ONLY the admin can add staff
router.post('/customers', staffOrAdmin, (req, res) => createWithRole('CUSTOMER', req,
router.post('/staff', adminOnly, (req, res) => createWithRole('STAFF', req, res));

router.delete('/staff/:id', adminOnly, async (req, res) => {
  const target = await prisma.user.findFirst({ where: { id: req.params.id, role: 'STAF
  if (!target) return res.status(404).json({ error: 'Staff member not found' });
  try {
    await prisma.user.delete({ where: { id: target.id } });
    res.json({ ok: true });
  } catch {
    res.status(400).json({ error: 'Cannot delete: this person has related records' });
  }
});

export default router;
```

`server/src/routes/me.js` **(customer portal)**

Never trust a customer id from the browser. Always use `req.user.id`.

```js
import { Router } from 'express';
import { prisma } from '../index.js';
import { requireAuth, requireRole } from '../middleware/auth.js';

const router = Router();
const customerOnly = [requireAuth, requireRole('CUSTOMER')];
const VISIBLE = ['CONFIRMED', 'PAID']; // customers never see draft or cancelled invoi

const derive = (inv) => {
  const outstanding = Math.round((Number(inv.total) - Number(inv.amountPaid)) * 100) /
  return { ...inv, outstanding, isOverdue: inv.status === 'CONFIRMED' && inv.dueDate <
};

router.get('/subscriptions', customerOnly, async (req, res) => {
  const items = await prisma.subscription.findMany({
    where: { customerId: req.user.id, status: { not: 'DRAFT' } },
    include: { plan: true, lines: { include: { product: true, variant: true } } },
    orderBy: { createdAt: 'desc' },
  });
  res.json({ items });
});

router.get('/invoices', customerOnly, async (req, res) => {
  const page = Number(req.query.page) || 1;
  const limit = Math.min(Number(req.query.limit) || 10, 100);
  const where = { customerId: req.user.id, status: { in: VISIBLE } };
  const [items, total] = await Promise.all([
    prisma.invoice.findMany({
      where, include: { subscription: { select: { number: true } } },
      orderBy: { issueDate: 'desc' }, skip: (page - 1) * limit, take: limit,
    }),
    prisma.invoice.count({ where }),
  ]);
  res.json({ items: items.map(derive), total, page, pages: Math.ceil(total / limit) })
});

router.get('/invoices/:id', customerOnly, async (req, res) => {
  const inv = await prisma.invoice.findFirst({
    where: { id: req.params.id, customerId: req.user.id, status: { in: VISIBLE } },
    include: { lines: true, payments: { orderBy: { date: 'desc' } }, subscription: { s
  });
  if (!inv) return res.status(404).json({ error: 'Invoice not found' }); // 404, not 4
  res.json(derive(inv));
});


export default router;
```

These routes use the `Subscription` and `Invoice` tables from Persons 3 and 4. Until their schemas are merged, **write the file but don't mount it**, and build the portal screens with empty arrays.

`server/src/index.js` : add these

```js
```

```
import authRoutes from './routes/auth.js';
import userRoutes from './routes/users.js';
import meRoutes from './routes/me.js';

if (!process.env.JWT_SECRET) throw new Error('JWT_SECRET is missing in .env');

app.use('/api/auth', authRoutes);
app.use('/api/users', userRoutes);
app.use('/api/me', meRoutes); // mount this after Persons 3 and 4 have merged their sc

// put these AFTER all the app.use('/api/...') lines, right before app.listen
app.use('/api', (req, res) => res.status(404).json({ error: 'Route not found' }));
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({ error: 'Something went wrong' });
});
```

The error handler matters for everyone, because without it one unexpected error returns an ugly HTML page. If a teammate's error handler or 404 line already exists, keep only one copy.

`server/prisma/createAdmin.js` (so there's a first login)

js

```
import 'dotenv/config';
import { PrismaClient } from '@prisma/client';
import bcrypt from 'bcryptjs';

const prisma = new PrismaClient();
const passwordHash = await bcrypt.hash('Password@123', 10);

await prisma.user.upsert({
  where: { email: 'admin@brightloop.com' },
  update: { passwordHash, role: 'ADMIN' },
  create: { email: 'admin@brightloop.com', name: 'Admin', passwordHash, role: 'ADMIN'
});
console.log('Admin ready: admin@brightloop.com / Password@123');
await prisma.$disconnect();
```

Add `"create-admin": "node prisma/createAdmin.js"` to the scripts in `server/package.json`. Person 4's seed also creates this same admin, so both work.

3. Frontend shared pieces

`client/src/api/client.js`: replace with this

js

```
const BASE = 'http://localhost:5000/api';

export async function api(path, options = {}) {
  const token = localStorage.getItem('token');
  const res = await fetch(BASE + path, {
    ...options,
    headers: {
      'Content-Type': 'application/json',
      ...(token && { Authorization: `Bearer ${token}` }),
      ...options.headers,
    },
    body: options.body ? JSON.stringify(options.body) : undefined,
  });
  const data = await res.json().catch(() => ({}));
  // expired or invalid session: send them to login (but not for the login call itself
  if (res.status === 401 && !path.startsWith('/auth/')) {
    localStorage.clear();
    window.location = '/login';
    throw new Error('Session expired');
  }
  if (!res.ok) throw new Error(data.error || 'Request failed');
  return data;
}
```

client/src/utils/validators.js

These rules mirror the server, and Person 1's forms and the staff form use them.

js

```
export const passwordRules = [
  ['More than 8 characters', (p) => p.length > 8],
  ['An uppercase letter', (p) => /[A-Z]/.test(p)],
  ['A lowercase letter', (p) => /[a-z]/.test(p)],
  ['A special character', (p) => /[^A-Za-z0-9]/.test(p)],
];

export const isStrongPassword = (p) => passwordRules.every(([, test]) => test(p));
export const isEmail = (e) => /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(e);
```

client/src/components/PasswordChecklist.jsx

jsx

```
import { Check, X } from 'lucide-react';
import { passwordRules } from '../utils/validators';

export default function PasswordChecklist({ password }) {
  return (
    <ul className="mb-3 space-y-1 text-xs">
      {passwordRules.map(([label, test]) => {
        const ok = test(password);
        return (
          <li key={label} className={`flex items-center gap-1 ${ok ? 'text-green-600'
            {ok ? <Check size={14} /> : <X size={14} />} {label}
          </li>
        );
      })}
    </ul>
  );
}
```

client/src/routes/ProtectedRoute.jsx

jsx

```
import { Navigate, Outlet } from 'react-router-dom';

export const homeFor = (role) => (role === 'CUSTOMER' ? '/portal/subscriptions' : '/da

export const getUser = () => {
  try { return JSON.parse(localStorage.getItem('user')); } catch { return null; }
};

export default function ProtectedRoute({ roles }) {
  const user = getUser();
  if (!localStorage.getItem('token') || !user) return <Navigate to="/login" replace />
  if (roles && !roles.includes(user.role)) return <Navigate to={homeFor(user.role)} re
  return <Outlet />;
}
```

Remember: hiding pages in the browser is only for convenience. The real protection is the backend `requireRole`, which is why every route is checked there too.

4. Auth pages (`client/src/pages/auth/`)

Give all four a centred white card on a `bg-slate-50` page with the Brightloop name on top (same indigo theme). They're outside the Layout, so no sidebar.

`Login.jsx`

jsx

```
import { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { api } from '../../api/client';
import { homeFor } from '../../routes/ProtectedRoute';
import { isEmail } from '../../utils/validators';
import FormField, { inputClass } from '../../components/FormField';

export default function Login() {
  const navigate = useNavigate();
  const [form, setForm] = useState({ email: '', password: '' });
  const [error, setError] = useState('');
  const [busy, setBusy] = useState(false);

  const submit = async (e) => {
    e.preventDefault();
    setError('');
    if (!isEmail(form.email)) return setError('Enter a valid email address');
    if (!form.password) return setError('Password is required');
    setBusy(true);
    try {
      const { token, user } = await api('/auth/login', { method: 'POST', body: form })
      localStorage.setItem('token', token);
      localStorage.setItem('user', JSON.stringify(user));
      navigate(homeFor(user.role), { replace: true });
    } catch (err) { setError(err.message); }
    setBusy(false);
  };

  return (
    <div className="flex min-h-screen items-center justify-center bg-slate-50 p-4">
      <form onSubmit={submit} className="w-full max-w-sm rounded-xl border bg-white p-
        <h1 className="mb-1 text-2xl font-bold text-indigo-600">Brightloop</h1>
        <p className="mb-5 text-sm text-slate-500">Log in to your account</p>
        <FormField label="Email">
          <input type="email" className={inputClass} value={form.email} onChange={(e)
        </FormField>
        <FormField label="Password">
          <input type="password" className={inputClass} value={form.password} onChange
        </FormField>
        {error && <p className="mb-3 text-sm text-red-600">{error}</p>}
        <button disabled={busy} className="w-full rounded-lg bg-indigo-600 py-2 text-w
          {busy ? 'Logging in...' : 'Log in'}
        </button>
        <div className="mt-4 flex justify-between text-sm">
          <Link to="/forgot-password" className="text-indigo-600">Forgot password?</Li
          <Link to="/register" className="text-indigo-600">Create account</Link>
        </div>
      </form>
    </div>
  );
}
```

`FormField` and `inputClass` come from Person 2's shared components. If they aren't merged yet, copy them locally for now and don't commit a duplicate.

`Register.jsx`

Same card style. Fields: name, email, password, confirm password.

- Show `<PasswordChecklist password={form.password} />` under the password field, so it ticks live.
- Before submitting, check: name has at least 2 characters, `isEmail`, `isStrongPassword`, and the two passwords match ("Passwords do not match").
- `POST /auth/register` with `{ name, email, password }`. The server returns a token and user, so store them exactly like Login and navigate to `/portal/subscriptions`.
- If the server says the email already exists, show it under the form ("An account with this email already exists").
- Link back to login.

`ForgotPassword.jsx`

One email field. On submit, `POST /auth/forgot-password`, then replace the form with the server's message ("If that email is registered, a reset link has been sent."). Link back to login.

`ResetPassword.jsx`

- Read the token: `const token = new URLSearchParams(useLocation().search).get('token')`. If it's missing, show "Invalid reset link".
- Fields: new password (with `PasswordChecklist`) and confirm.
- `POST /auth/reset-password` with `{ token, password }`, then show a success message and navigate to `/login` after 2 seconds. Show the server error for expired or used links.

5. `App.jsx`: restructure it (do this in your first push)

This is how every role gets locked to its own pages. The other 3 keep their own `<Route>` lines, and only the wrapping changes.

```jsx
```

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import Layout from './components/Layout';
import ProtectedRoute, { getUser, homeFor } from './routes/ProtectedRoute';
import Login from './pages/auth/Login';
import Register from './pages/auth/Register';
import ForgotPassword from './pages/auth/ForgotPassword';
import ResetPassword from './pages/auth/ResetPassword';
import Staff from './pages/admin/Staff';
import Customers from './pages/staff/Customers';
import MySubscriptions from './pages/portal/MySubscriptions';
import MyInvoices from './pages/portal/MyInvoices';
import MyInvoiceDetail from './pages/portal/MyInvoiceDetail';

const Placeholder = ({ name }) => <h2 className="text-2xl font-semibold">{name}</h2>;

export default function App() {
  const user = getUser();
  return (
    <BrowserRouter>
      <Routes>
        {/* public */}
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />
        <Route path="/forgot-password" element={<ForgotPassword />} />
        <Route path="/reset-password" element={<ResetPassword />} />

        {/* must be logged in */}
        <Route element={<ProtectedRoute />}>
          <Route element={<Layout />}>

            {/* admin + staff */}
            <Route element={<ProtectedRoute roles={['ADMIN', 'STAFF']} />}>
              <Route path="/dashboard" element={<Placeholder name="Dashboard" />} />
              <Route path="/customers" element={<Customers />} />
              <Route path="/subscriptions" element={<Placeholder name="Subscriptions"
              <Route path="/invoices" element={<Placeholder name="Invoices" />} />
              <Route path="/payments" element={<Placeholder name="Payments" />} />
            </Route>

            {/* admin only */}
            <Route element={<ProtectedRoute roles={['ADMIN']} />}>
              <Route path="/products" element={<Placeholder name="Products" />} />
              <Route path="/plans" element={<Placeholder name="Plans" />} />
              <Route path="/templates" element={<Placeholder name="Templates" />} />
              <Route path="/taxes" element={<Placeholder name="Taxes" />} />
              <Route path="/discounts" element={<Placeholder name="Discounts" />} />
              <Route path="/staff" element={<Staff />} />
            </Route>

            {/* customers only */}
            <Route element={<ProtectedRoute roles={['CUSTOMER']} />}>
              <Route path="/portal/subscriptions" element={<MySubscriptions />} />
              <Route path="/portal/invoices" element={<MyInvoices />} />
              <Route path="/portal/invoices/:id" element={<MyInvoiceDetail />} />
            </Route>

          </Route>
        </Route>

        <Route path="*" element={<Navigate to={user ? homeFor(user.role) : '/login'} r
      </Routes>
    </BrowserRouter>
  );
}
```

The `/subscriptions/new` and `/subscriptions/:id` routes (Person 3) and `/invoices/:id`

(Person 4) go inside the admin+staff block. Tell them to add their extra routes there. **Push this first and tell everyone**, because they were told to edit `App.jsx` and this changes where their lines live.

`Layout.jsx` : small edits

- Remove the fallback: `const user = JSON.parse(localStorage.getItem('user'))` (ProtectedRoute guarantees a user now).
- Add `print:hidden` to the `<aside>` and `<header>` classes so printed invoices skip them (Person 4 asked for this).
- Add Templates (`['/templates', 'Templates', FileStack]` , importing `FileStack` from `lucide-react`) to the ADMIN menu if Person 2 hasn't already.
- Keep Logout clearing localStorage and sending to `/login` .

Check with Persons 2 and 4 first, because both were told to edit `Layout.jsx` . Whoever does it first wins, and the others skip it after pulling.

6. Staff and customer pages

`Staff.jsx` (admin only, `/staff`)

Same structure as Person 2's `Products.jsx` :

- Table (`GET /users?role=STAFF&page=&search=`): Name, Email, Created, and a delete icon. Search box and pagination.
- **Add staff** button opens a `Modal` with name, email, password (+ `PasswordChecklist`). Validate with `isEmail` and `isStrongPassword` before calling `POST /users/staff` . Show server errors like "An account with this email already exists".
- Delete asks `confirm()` and calls `DELETE /users/staff/:id` .

`Customers.jsx` (admin and staff, `/customers`)

Same pattern with `GET /users?role=CUSTOMER` . Columns: Name, Email, Joined. Add a **New customer** modal with name, email and a temporary password (`POST /users/customers`). Staff can add customers but cannot see or add staff.

7. Customer portal (`client/src/pages/portal/`)

Customers see only their own data, and the layout shows only the two portal menu items.

- `MySubscriptions.jsx` : `GET /me/subscriptions` . Show one card per subscription: number, `StatusBadge` (from Person 3), plan name and billing period, start and expiry dates, and a small table of lines (product, variant, quantity, unit price). Empty state: "You have no subscriptions yet."
- `MyInvoices.jsx` : `GET /me/invoices?page=` . Use the shared `Table` : Number, Issue date, Due date, Total, Outstanding, Status. Show an `OVERDUE` badge when `isOverdue` is true. Each row links to `/portal/invoices/:id` . Show a small summary on top: total outstanding and the number overdue, calculated from the list.
- `MyInvoiceDetail.jsx` : `GET /me/invoices/:id` . Show header info, lines table (description, qty, unit price, discount, tax, total), totals box (subtotal, discount, tax, total, paid, outstanding), and payments history. A **Print** button (`window.print()` , wrapped in `print:hidden`). There are no staff actions here.

Use responsive styles, because customers will open this on a phone. Tables go inside an `overflow-x-auto` container, which the shared `Table` already has.

8. Contracts with the others (don't change without telling them)

- `requireAuth` and `requireRole('ADMIN', 'STAFF')` are exported from `server/src/middleware/auth.js` . After `requireAuth` , `req.user` is `{ id, name, email, role }` .
- The token and user are saved in `localStorage` as `token` and `user` (`{ id, name, email, role }`), and the `api()` helper adds the `Authorization` header automatically.
- **Person 3 asked for:** `GET /api/users?role=CUSTOMER&limit=100` , which returns `{ items, total, page, pages }` . Tell them the shape.
- **Roles:** admin does everything. Staff can read most things and manage subscriptions, invoices, payments and customers, but cannot create products, plans, taxes, discounts or staff. Customers only touch `/api/me/*` .
- `ProtectedRoute` and `homeFor` are the only role-routing helpers. Nobody should build their own.

9. Test it properly

1. **Signup rules:** try each one failing alone: 8 characters exactly, no uppercase, no lowercase, no special character. Each should show a message. Then try a duplicate email (error), and success.

2. **Forgot password:** request a reset for a real and a fake email. Both show the same message. For the real one, copy the link from the server console and reset. Try the same link again: it must fail (single use).
3. **Roles in the browser:** log in as a customer and type `/products` in the address bar. You should be bounced to the portal.
4. **Roles at the API (the important one):** with a customer token, call `GET /api/subscriptions` and `POST /api/users/staff`. Both must return **403**. With a staff token, `POST /api/users/staff` must return **403**. With no token, any `/api/...` route (except auth) must return **401**.
5. **Data isolation:** log in as `customer1@example.com` and `customer2@example.com` (after the seed). Copy an invoice id from customer 1 and open `/api/me/invoices/<that id>` as customer 2. It must return 404.
6. **Staff creation:** only the admin sees Staff in the menu and can add staff.
7. A customer must not see DRAFT or CANCELLED invoices.

10. Rules

- The server decides permissions. The UI hiding things is only a convenience.
- Register always creates `CUSTOMER`. Never read `role` from the request body.
- Passwords are only ever stored as bcrypt hashes and never returned in a response.
- Login failures use one message for "no such email" and "wrong password".
- Emails are lowercased before saving, so `A@x.com` and `a@x.com` count as the same account.

11. Checklist

- [ ] `auth.js` middleware pushed in the first hour and teammates told to drop their stub
- [ ] `App.jsx` restructured and pushed early
- [ ] `npm run create-admin` works, and admin can log in
- [ ] Register with live password checklist and unique email
- [ ] Login, logout, and expired session handling
- [ ] Forgot and reset password (link in console, and email if SMTP is set up)
- [ ] Admin can add and delete staff, and staff cannot
- [ ] Staff and admin can add customers
- [ ] Portal: subscriptions, invoices, invoice detail with print
- [ ] All 7 tests in section 9 pass
- [ ] Mobile width looks fine

12. Git

```bash
git add .
git commit -m "feat: auth, roles, staff and customers, portal"
git push origin feature/p1-auth
```

Push small and often: middleware, `App.jsx`, auth routes, auth pages, reset flow, staff and customers, then the portal. Open the first PR (middleware, `App.jsx` and `createAdmin`) as early as you can, because three people are waiting on it.

Your job after finishing: you're the integration lead. Update the README with the run steps and the login accounts, check each PR for the role protections, and run the full demo script from the brief once before the deadline. If you hit a bug, paste the error and the file to me or your AI tool.